

AI 제조 기술 관련 교육

『3과목』 최소구현 + 리뷰

2025.12.08-2025.12.19(10일 70시간)

Prepared by Daekyeong K

Ph.D.

SAMSUNG DISPLAY



한국기술교육대학교

KORATECH

Copyright 2022. Daekyeong all rights reserved

『3과목』 최소구현 + 리뷰

- Part 1. 오리엔테이션
- Part 2. GREEN, Review Cursor AI 프롬프트
- Part 3. SW Implementation
- Part 4. GREEN 단계의 최소 구현
- Part 5. 종합 실습 & 마무리





1. 학습 개요

학습목표

- 이 워크샵에서는 코드에서 설계 문서를 역추적하는 실전 가이드에 대해 알 수 있습니다.
- 비즈니스 목표
 - 개발 생산성 30% 향상 (문서화 자동화)
 - 협업 효율성 증대 (팀원 온보딩 시간 50% 단축)
 - 유지보수 비용 절감 (설계-코드 일치율 95% 이상)
- 기술 목표
 - ERD, API 명세서, 아키텍처 다이어그램 자동 생성
 - 설계-구현 일치 검증 프로세스 구축
 - CI/CD 파이프라인에 문서 자동화 통합



1. 학습 개요

산출물

- 실무 표준 ERD (Mermaid + 주석)
- OpenAPI 3.0 명세서 (Swagger UI)
- 아키텍처 문서 (C4 Model 적용)
- 통합 README.md (프로젝트 온보딩 가이드)
- 검증 보고서 (설계-구현 일치율)



2. 실무에서의 설계 문서화 현실



실무 현황 및 문제점

- 현장 조사 결과 (2024 한국 SW 개발팀 설문)
 - 설계 문서 작성률: 34% (3개 팀 중 1개만 작성)
 - 문서 작성 시간: 전체 개발 시간의 15-20%
 - 문서-코드 불일치율: 67%
 - 6개월 후 문서 유효성: 22%



2. 실무에서의 설계 문서화 현실

실무 Pain Point

● 목표

- AI 자동화로 문서 작성 시간 80% 단축
- 설계-코드 일치율 95% 이상 달성

문제	영향	비용
설계 문서 없음	신입 온보딩 4주 → 8주	인건비 2배
문서-코드 불일치	버그 수정 시간 3배 증가	기회비용 손실
레거시 코드 이해 불가	리팩토링 불가능	기술 부채 누적
API 문서 누락	프론트-백엔드 협업 지연	일정 지연



3. OrderBean 프로젝트 개요

| 비즈니스 요구사항

- 카페 주문 관리 시스템
- 회원가입, 로그인, 메뉴 조회, 주문, 쿠폰 적용



3. OrderBean 프로젝트 개요

기술 스택

Backend:

- FastAPI 0.109.0
- SQLAlchemy 2.0
- PostgreSQL / SQLite

Testing:

- Pytest 7.4
- Coverage 100% 목표

Documentation:

- Mermaid (다이어그램)
- OpenAPI 3.0 (API 명세)
- Markdown (문서)

Deployment:

- Docker
- Render / AWS
- GitHub Actions (CI/CD)



3. OrderBean 프로젝트 개요

| Day 1-2 완료 상태 체크 리스트

- PRD 작성 완료
- 클린코드 + TDD 구현 완료
- User, Order, Menu, Coupon 모델 완성
- 주요 API 엔드포인트 8개 구현
- 테스트 커버리지 85%

『3과목』 최소구현 + 리뷰

- Part 1. 오리엔테이션
- Part 2. GREEN, Review Cursor AI 프롬프트
- Part 3. SW Implementation
- Part 4. GREEN 단계의 최소 구현
- Part 5. 종합 실습 & 마무리





1. GREEN 단계 - - 최소 구현으로 테스트 통과.

GREEN 단계의 목표

- "테스트를 통과하는 가장 간단한 코드를 작성한다"

- ✓ 모든 테스트가 통과해야 함
- ✓ 테스트 통과에 필요한 최소한의 코드만 작성
- ✗ 과도한 최적화 금지
- ✗ 추가 기능 구현 금지
- ✗ "나중에 필요할 것 같은" 코드 작성 금지



1. GREEN 단계 - - 최소 구현으로 테스트 통과.

Cursor AI 프롬프트 템플릿 (GREEN)

Cursor AI 프롬프트 - GREEN 단계

컨텍스트

- 테스트 파일: [테스트 파일 경로]
- 구현 파일: [구현할 파일 경로]

테스트 코드

```
```typescript  
[전체 테스트 코드 붙여넣기]
```
```

요청

위 테스트를 ****모두 통과****하는 ****최소한의 구현 코드****를 작성해주세요.

요구사항

1. 테스트 통과에 필요한 코드만 작성
2. 과도한 최적화나 추가 기능 금지
3. TypeScript 타입 완전 정의
4. Props 인터페이스 명확히 정의

출력 형식

1. 타입 정의 (필요시)
2. 컴포넌트/함수 구현
3. CSS 스타일 (필요시)



1. GREEN 단계- - 최소 구현으로 테스트 통과.

GREEN 단계 작업 순서

Step 1: 테스트 코드 확인



Step 2: Cursor AI에 프롬프트 입력



Step 3: 생성된 코드 확인 및 파일 생성



Step 4: 테스트 실행 및 통과 확인



Step 5: 실패 시 수정, 성공 시 다음 단계



1. GREEN 단계 - - 최소 구현으로 테스트 통과.

GREEN 단계 주의사항

✓ DO (해야 할 것)

- 테스트 코드 전체를 Cursor에 제공
- 타입 정의 먼저 확인
- 컴포넌트/함수 하나씩 구현
- 테스트 실행하여 통과 확인
- 실패한 테스트에 집중



1. GREEN 단계 - - 최소 구현으로 테스트 통과.

GREEN 단계 주의사항

✗ DON'T (하지 말아야 할 것)

- 테스트에 없는 기능 추가
- "더 좋은 방법"으로 변경
- 미리 성능 최적화
- 불필요한 에러 처리 추가
- 테스트 없이 코드 수정



2. Review - 코드 리뷰 및 개선 포인트 도출

Review 단계의 목표

"구현된 코드의 품질을 객관적으로 분석하고 개선 포인트를 도출한다"

- 코드 품질 분석 (가독성, 유지보수성)
- 성능 이슈 식별
- 보안 취약점 확인
- 테스트 커버리지 검토
- 개선 포인트 우선순위화



2. Review - 코드 리뷰 및 개선 포인트 도출

Cursor AI 프롬프트 템플릿 (Review)

Cursor AI 프롬프트 - Review 단계

컨텍스트

- 구현 코드: [구현 파일 경로]
- 테스트 코드: [테스트 파일 경로]

구현 코드

```
```typescript
[구현 코드 붙여넣기]
```
```

요청

코드 리뷰를 진행하고 다음을 분석해주세요:

리뷰 관점

1. **코드 품질**: 가독성, 유지보수성
2. **성능**: 불필요한 렌더링, 메모리 누수
3. **타입 안정성**: TypeScript 활용도
4. **테스트 커버리지**: 누락된 테스트
5. **접근성**: a11y 준수 여부
6. **보안**: XSS, 입력 검증

출력 형식

| 영역 | 현재 상태 | 개선 제안 | 우선순위 |
|-------|-------|-------|----------|
| ----- | ----- | ----- | ----- |
| 성능 | ... | ... | P0/P1/P2 |

개선 포인트 코드 예시

각 제안에 대한 구체적인 코드 예시를 포함해주세요.



2. Review - 코드 리뷰 및 개선 포인트 도출

리뷰 관점 상세 설명

1) 코드 품질 (가독성, 유지보수성)

검토 항목:

- 변수/함수 명명 규칙
- 함수 길이 (20줄 이내 권장)
- 단일 책임 원칙 (SRP)
- 중복 코드 여부
- 주석 필요성



2. Review - 코드 리뷰 및 개선 포인트 도출

리뷰 관점 상세 설명

2) 성능

검토 항목:

- 불필요한 리렌더링 (React.memo, useMemo, useCallback)
- 메모리 누수 (useEffect cleanup)
- 무한 루프 가능성
- N+1 쿼리 문제 (Backend)
- 불필요한 API 호출



2. Review - 코드 리뷰 및 개선 포인트 도출

리뷰 관점 상세 설명

3) 타입 안정성

검토 항목:

- any 타입 사용 여부
- 제네릭 활용도
- 타입 가드 사용
- 옵셔널 체이닝 활용
- null/undefined 처리



2. Review - 코드 리뷰 및 개선 포인트 도출

리뷰 관점 상세 설명

4) 테스트 커버리지

검토 항목:

- 엣지 케이스 테스트 여부
- 에러 케이스 테스트 여부
- 비동기 작업 테스트
- 모킹 적절성
- 테스트 격리



2. Review - 코드 리뷰 및 개선 포인트 도출

리뷰 관점 상세 설명

5) 접근성 (a11y)

검토 항목:

- aria-label 사용
- 키보드 네비게이션
- 색상 대비
- 스크린 리더 지원
- 포커스 관리



2. Review - 코드 리뷰 및 개선 포인트 도출

리뷰 관점 상세 설명

6) 보안

검토 항목:

- XSS 공격 방지
- 입력 검증
- SQL 인젝션 방지 (Backend)
- CSRF 토큰
- 민감 정보 노출



2. Review - 코드 리뷰 및 개선 포인트 도출

리뷰 관점 상세 설명

우선순위 기준

| 우선순위 | 의미 | 조치 시점 |
|------|-----------------------|---------------|
| P0 | 즉시 수정 필요 (버그, 보안 취약점) | 지금 당장 |
| P1 | 높은 우선순위 (성능, 가독성) | Refactor 단계에서 |
| P2 | 낮은 우선순위 (개선 사항) | 시간 여유 시 |

『3과목』 최소구현 + 리뷰

- Part 1. 오리엔테이션
- Part 2. GREEN, Review Cursor AI 프롬프트
- Part 3. SW Implementation
- Part 4. GREEN 단계의 최소 구현
- Part 5. 종합 실습 & 마무리





1. “소프트웨어 개발(Development)”이란?

소프트웨어 개발을 위해 알아야 할 것들

① Computer Science 기본기 (필수 기반)

기초 CS 지식은 개발 역량의 “하한”과 “상한”을 결정합니다.

● 알아야 할 개념

- 자료구조(Data Structures)
 - 배열, 리스트, 스택, 큐, 해시맵, 그래프, 트리
 - → 어떤 문제를 어떤 자료구조로 해결할지 판단하는 능력
- 알고리즘(Algorithms)
 - 탐색, 정렬, 분할정복, DP, 그리디
 - Big-O 시간/공간 복잡도
- 운영체제(OS)
 - 프로세스/스레드, 메모리 구조, 동기화/경쟁 상태, 파일 시스템
 - → 멀티스레딩, async/await를 이해하려면 필수
- 네트워크(Networking)
 - TCP/IP, HTTP, DNS, REST, WebSocket
 - → API 설계와 서버 개발의 근본
- 데이터베이스(DB)
 - SQL, 트랜잭션(ACID), 인덱스, 조인, 정규화
 - NoSQL 특성 이해 (MongoDB, Redis, ElasticSearch)



1. “소프트웨어 개발(Development)”이란?

소프트웨어 개발을 위해 알아야 할 것들
하드웨어 및 네트워크 환경을

| 업무명 | 장비명 | 수량 | 용도 | 보유형태 | 구입시기 | 설치장소 | 플랫폼 |
|-----------|-------|----|-----|------|------|------|--------|
| 내부 사용자 DB | G60 | 1 | DB | 구입 | 2022 | 서버실 | HP-UX |
| 외부 포털 서비스 | ML530 | 1 | WEB | 구입 | 2022 | DMZ | Maria |
| LDAP | DL510 | 1 | SSO | 구입 | 2022 | 서버실 | Window |

출처:

정보통신산업진흥원(2009). 정보 시스템 마스터플랜(ISMP) 방법론.



1. “소프트웨어 개발(Development)”이란?

소프트웨어 개발을 위해 알아야 할 것들 인프라 환경

- Web/App. Server :

| |
|---------------------------------|
| eGovFramework(전자정부) |
| UI : Websquare/Nexacro |
| Reporting Tool |
| WAS:JEUS/Tomcat |
| Unix/Linux/WindowsServer |
| TCP/IP |

- DB Server :

| |
|---------------------------------|
| Oracle/MS-SQL |
| Unix/Linux/WindowsServer |
| TCP/IP |



1. “소프트웨어 개발(Development)”이란?

소프트웨어 개발을 위해 알아야 할 것들

② 프로그래밍 언어 역량 (언어 1개는 깊게, 1~2개는 넓게)

- 깊게 파야 할 요소
 - 언어의 메모리 모델
 - 타입 시스템
 - 예외 처리
 - OOP vs FP
 - 디자인 패턴 (GoF)
 - 모듈/패키지 구조 설계
 - 테스트 코드 (xUnit)
 - 리팩토링/클린코드
- 추천 언어 조합
 - Python + Java → 실무 만능 조합
 - C# → Windows + 엔터프라이즈 + 게임까지 커버
 - JavaScript/TypeScript → 웹·프론트엔드 필수



1. “소프트웨어 개발(Development)”이란?

소프트웨어 개발을 위해 알아야 할 것들

③ SW 설계 및 아키텍처 (Junior → Mid → Senior로 성장하는 핵심)

- 핵심 설계 스킬
 - 객체지향 설계(OOP): SRP / OCP / ISP / LSP / DIP
 - 클린 아키텍처
 - DDD (Domain-Driven Design)
 - Layered Architecture / Hexagonal Architecture
 - REST API 설계
 - Microservice 개념
 - Event-driven architecture (EDA)
- 코드 레벨에서는?
 - 코드 스멜 감지
 - 리팩토링 원칙
 - SOLID 적용
 - 테스트 주도 개발(TDD)
 - 예외 처리 전략
 - 유지보수 가능한 구조 설계



1. “소프트웨어 개발(Development)”이란?

소프트웨어 개발을 위해 알아야 할 것들 소프트웨어 아키텍처의 정의

| 구분 | 정의 |
|--------------------|--|
| IEEE-1471 (국제 표준) | - 시스템에 대한 기본 조직 체계로 시스템을 이루는 구성 요소와 이 구성 요소들 사이의 관계, 구성 요소와 주변 환경과의 관계, 시스템의 진화 방향, 설계를 지배하는 원칙들로 실체화된다. |
| SEI(카네기멜론대 SW공학센터) | - 시스템을 구성하기 위해 필요한 구조의 집합이며, 이는 요소와 관계, 요소 속성과 관계 속성을 포함한다. |



1. “소프트웨어 개발(Development)”이란?

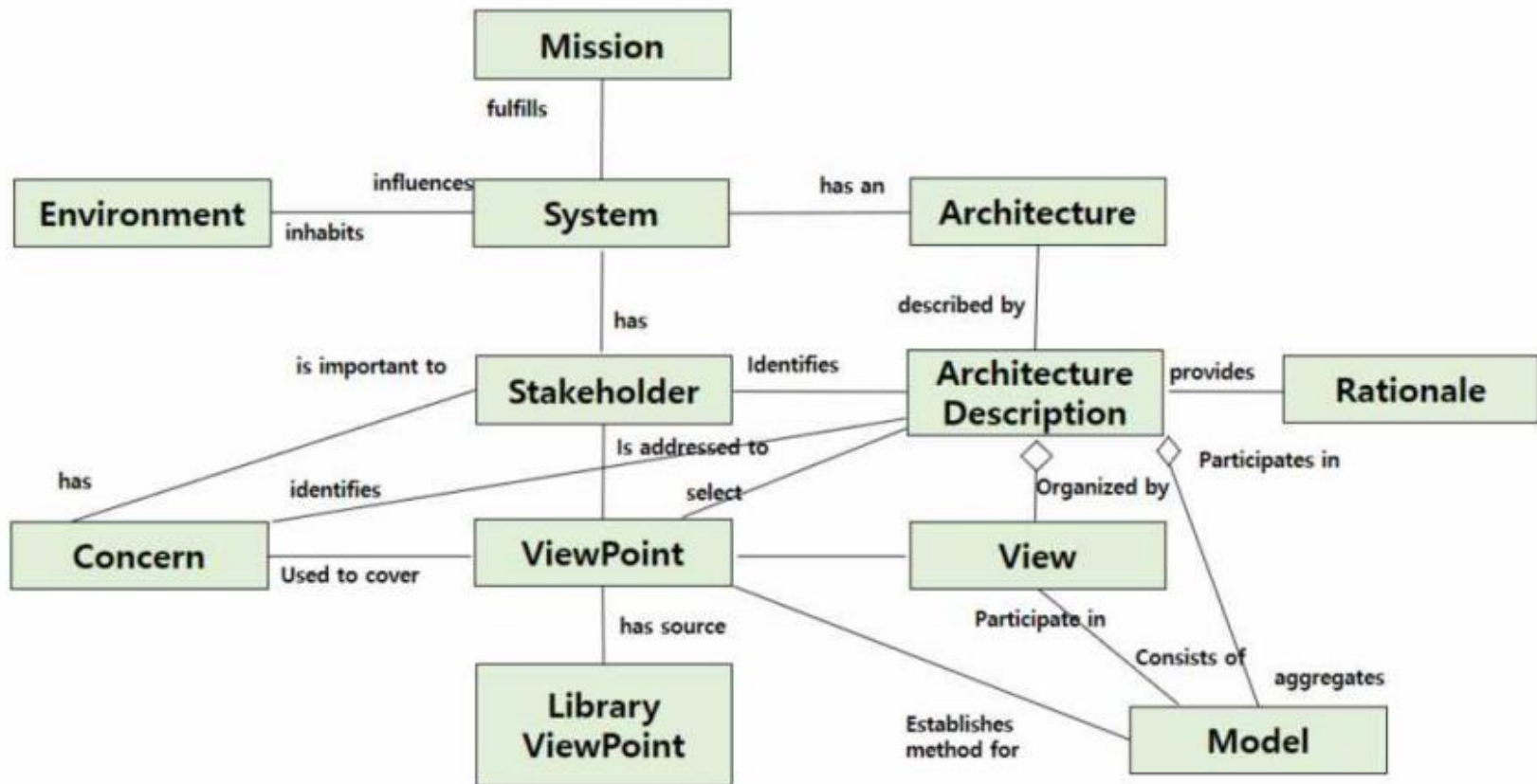
소프트웨어 개발을 위해 알아야 할 것들

소프트웨어 아키텍처의 구성 요소

| 구성 요소 | 설명 | 예시 |
|------------------|--|--------------------|
| 컴포넌트(아키텍처 구성 요소) | - 구체화된 시스템 기본 조직, 소프트웨어 개별 구성 요소 | - HW, SW, 모듈 등 |
| 관계 | - 각종 소프트웨어 컴포넌트 간의 관계, 이해관계자 관점(view)에 따라 관계를 설정 | - 연관, 집합, 포함 관계 등 |
| 원리(품질속성) | - 소프트웨어의 디자인, 진화를 이끄는 기본적인 이론(결정 셋) | - 제약사항, 패턴, 요구사항 등 |

1. “소프트웨어 개발(Development)”이란?

소프트웨어 개발을 위해 알아야 할 것들 IEEE-1471의 구성 요소





1. “소프트웨어 개발(Development)”이란?

소프트웨어 개발을 위해 알아야 할 것들 소프트웨어 아키텍처 뷰의 유형

| 아키텍처 뷰 | 개념도 | 설명 |
|------------------|-----|---|
| Simens Four View | | <ul style="list-style-type: none">- 개념 뷰 : 시스템 상위 컴포넌트 및 그들의 관계 식별- 모듈 뷰 : 시스템 분할과 레이어로 모듈을 나눔.- 코드 뷰 : 프로그램 소스 코드를 어떤 단위나 형식으로 구조화- 실행 뷰 : 시스템 런타임과 개체 속성을 정의 |
| SEI 3 View | | <ul style="list-style-type: none">- 모듈(Module) 뷰 : 모듈 형태 관계 표현- C&C(Component & Connector) 뷰 : 연결 형태의 다양화 표현- 할당(Allocation) 뷰 : SW 요소, 환경 요소(User, NW 등) 고려 표현 |



1. “소프트웨어 개발(Development)”이란?

소프트웨어 개발을 위해 알아야 할 것들 소프트웨어 아키텍처 뷰의 유형

4+1 View

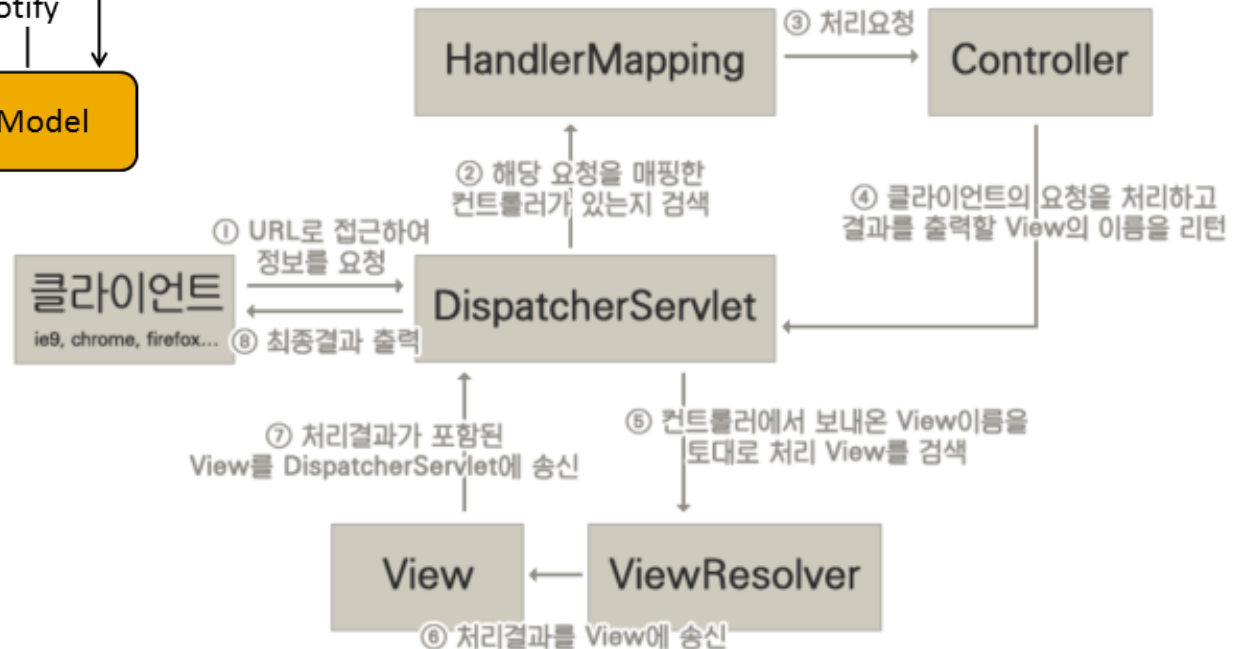
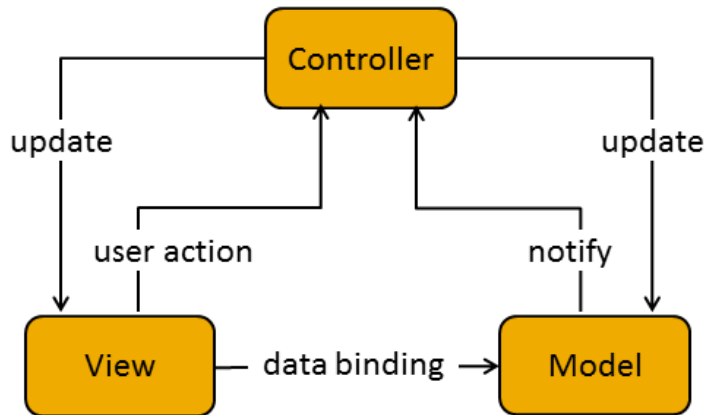


- **유스케이스(Use Case) 뷰** : 시스템 외부 사용자 관점에서 사용 사례와 이들 간의 관계를 정의, 시스템 아키텍처 도출
- **로지컬(Logical) 뷰** : 시스템에서 사용되는 관점에서 작업을 수행하는 논리적 컴포넌트와 관계 정의
- **프로세스(Process) 뷰** : 프로세스와 프로세서로 구분되는 시스템의 분할을 설명하고 비기능적 속성을 다루는 것으로 자원의 효율화, 이벤트 처리 등을 사용
- **구현(Implementation) 뷰** : 구현 컴포넌트 아키텍처 모델 정의
- **적용(Deployment) 뷰** : 물리적 시스템의 노드에 해당하는 물리적 컴포넌트의 대응



1. “소프트웨어 개발(Development)”이란?

소프트웨어 개발을 위해 알아야 할 것들 MVC 아키텍처와 Spring MVC 아키텍처

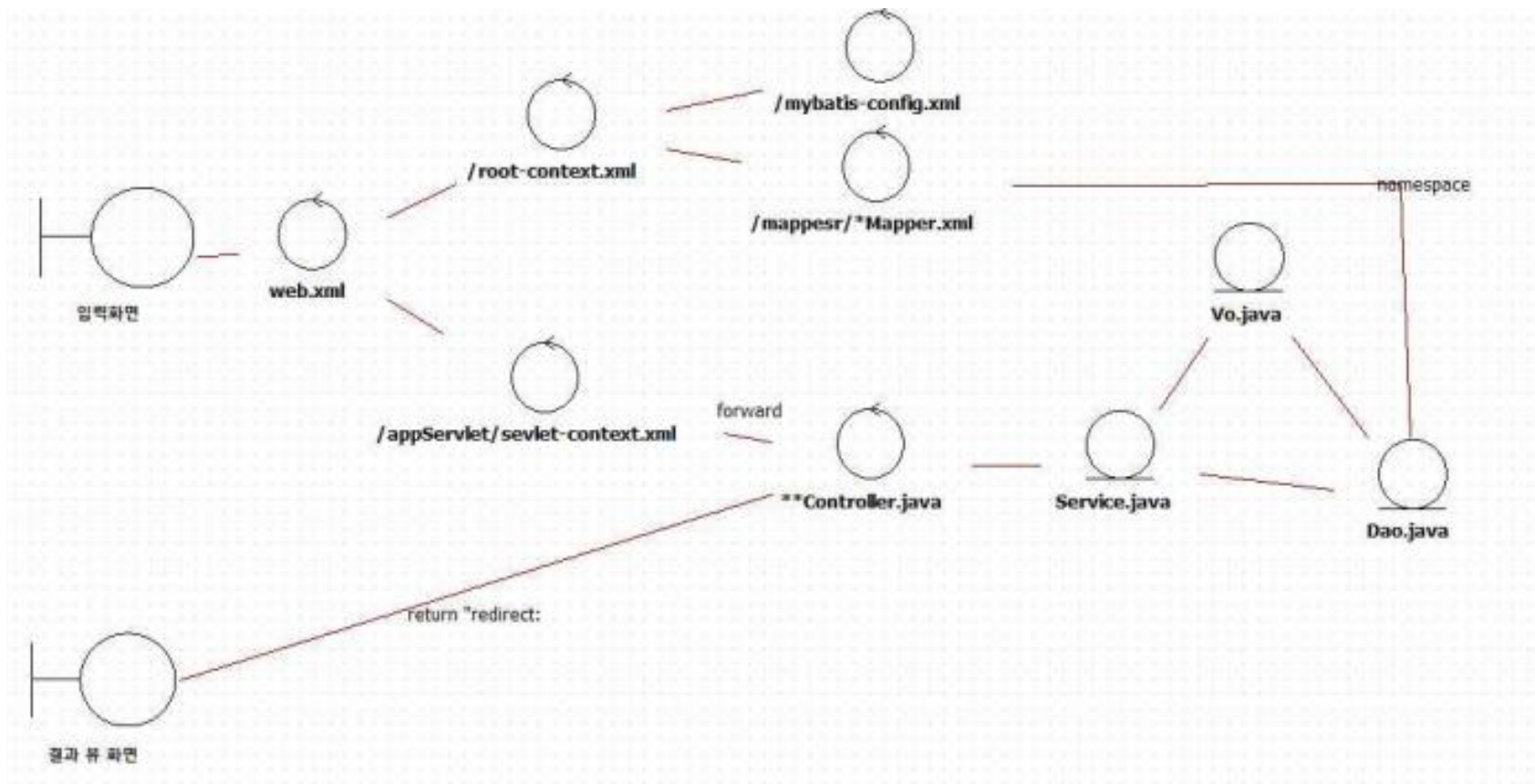




1. “소프트웨어 개발(Development)”이란?

소프트웨어 개발을 위해 알아야 할 것들

Spring MVC 아키텍처 개념 모델





1. “소프트웨어 개발(Development)”이란?

소프트웨어 개발을 위해 알아야 할 것들

Spring MVC 아키텍처 구성 요소 및 기술

| 구성요소 | 설명 |
|--------------------------|--|
| DispatcherServlet | 클라이언트의 요청을 받아 Controller에게 클라이언트의 요청을 전달하고, 결과 값을 View에게 전달하여 응답을 생성 |
| HandlerMapping | URL과 요청 정보를 기준으로 어떤 핸들러 객체를 사용할지 결정하는 객체. DispatcherServlet은 하나 이상의 핸들러 매핑을 가질 수 있다 |
| Controller | 클라이언트의 요청을 처리한 뒤 Model을 호출하고 결과를 DispatcherServlet에게 알려줌 |
| ModelAndView | Controller가 처리한 데이터 및 화면에 대한 정보를 보유한 객체, Controller의 리턴 타입 |
| View | Controller의 처리 결과 화면에 대한 정보를 보유한 객체 |
| ViewResolver | Controller가 리턴한 View 이름을 기반으로 Controller 처리 결과를 생성할 View를 결정. View를 쉽게 찾아서 클라이언트로 응답을 줄 수 있다 |



1. “소프트웨어 개발(Development)”이란? .

소프트웨어 개발을 위해 알아야 할 것들

④ 테스트(TDD), CI/CD, 자동화 역량 (현대 개발자의 필수 능력)

- 테스트 종류

- Unit Test
- Integration Test
- E2E Test
- Mock·Stub·Fake·Spy 개념
- Golden Master Test

- 자동화·배포(CI/CD)

- GitHub Action / GitLab CI / Jenkins
- Docker 필수
- Kubernetes는 선택 → 확장성 필요한 팀이라면 필수
- 자동 빌드/배포 파이프라인 구축



1. “소프트웨어 개발(Development)”이란?

소프트웨어 개발을 위해 알아야 할 것들

⑤ 실무형 개발 역량

- 필수 경험
 - API 서버 만들기
 - DB 모델링
 - 인증/인가(JWT, OAuth2)
 - 캐싱 전략
 - 성능 프로파일링 & 최적화
 - 로그 수집/분석
 - 클라우드(AWS, GCP, Azure)
- 개발 문화
 - Git 실무 브랜치 전략
 - 코드 리뷰 문화
 - 협업 툴 (Jira, Notion, Slack)



1. “소프트웨어 개발(Development)”이란? .

소프트웨어 개발을 위해 알아야 할 것들

⑥ 프론트엔드 & 사용자 경험

● 핵심 기술

- HTML / CSS / JavaScript
- React / Next.js
- API 연동
- 상태관리 (Redux, Zustand, Recoil 등)
- 성능 최적화(lazy loading, memoization)
- 반응형 UI
- 접근성(Accessibility)
- 디자인 시스템 구성(shadcn/ui, ChakraUI, MUI 등)



1. “소프트웨어 개발(Development)”이란? .

소프트웨어 개발을 위해 알아야 할 것들

⑦ AI 개발 역량 (현 시대의 필수)

● 필수 요소

- 머신러닝 기초, 딥러닝 프로그래밍
- 벡터 DB(FAISS, Chroma)
- 임베딩, 토큰나이저 개념
- RAG 파이프라인 구성
- LangChain / LlamaIndex / OpenAI API
- Whisper, TTS, OCR
- Multi-document RAG, 캐싱, reranker
- Agents(도구 기반 에이전트 설계)
- 생성형 모델 + 앱 개발(React, Streamlit, Gradio 등)

1. “소프트웨어 개발(Development)”이란? .

| 개발자 성장을 결정하는 5대 스킬셋

| 영역 | 설명 |
|--------------|---------------------------|
| 1. 문제 분해 능력 | 문제를 작은 단위로 쪼개는 사고 구조 |
| 2. 설계 능력 | 구조를 잡고 확장성·유지보수성을 예측하는 능력 |
| 3. 코드 품질 | 클린 코드, 테스트 코드, 리팩토링 |
| 4. 도구 사용 능력 | Git, Docker, CI/CD, 디버거 |
| 5. 협업·커뮤니케이션 | Issue 정의, PR 작성, 코드 리뷰 |



1. “소프트웨어 개발(Development)”이란? .

무엇을 먼저 하면 되는가?

- 1단계(바로 적용할 수 있는 실천)
 - Git 브랜치 전략 1가지 확립
 - 테스트 코드 습관화
 - 기능 → 테스트 → 리팩토링 루틴 구축
 - 안정적인 RAG 앱 1개 완성
 - DB 설계 패턴 정리(1:N / M:N / 중간 테이블)



1. “소프트웨어 개발(Development)”이란? .

무엇을 먼저 하면 되는가?

- 2단계(시니어 개발자 로드맵)
 - 클린 아키텍처 기반 프로젝트 1개
 - Refactoring 2nd Edition 완독
 - 오픈소스 1개 기여
 - CI/CD 자동화
 - Cloud + Docker 기반 운영 환경 구축



2. “소프트웨어 구현(Implementation)”이란?

구현(Implementation)이란?

- 개발 = 설계 + 코드 작성 + 구조화 + 테스트 + 배포
- 구현 = 실제로 동작하게 만드는 전 과정 (기능 완성 + 예외 처리 + 안정성 + 배포 + 운영까지 포함)

즉, 구현은 개발보다 더 넓은 개념이며 “프로덕션 레벨에서 돌아가는 소프트웨어를 만드는 것”입니다.

- 정리하면:
- 구현 = 개발 + 운영 환경 구축 + 배포 + 장애 대응 + 유지 보수



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

① 요구사항 → 기능 → 코드로 변환하는 기술

개발보다 구현은 요구사항 해석 능력이 훨씬 중요합니다.

- 기능 요구사항(Functional Requirements)
- 비기능 요구사항(Non-Functional Requirements)
 - 성능
 - 트래픽
 - 보안
 - 장애 대응
 - API 응답시간
 - 유지보수성
- 엣지 케이스(Edge Cases)
- 예외 처리 시나리오
- 입력 검증(Input Validation)
- 오류 로깅 / 재시도 전략



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소 요구 사항 분류와 ID 부여 규칙 (예시)

| 요구 사항 구분 | 영문 명칭 (Requirement Type) | ID 부여 규칙 | 요구 사항 수 |
|-----------------|---|-----------|---------|
| 기능 - SFR | System Function Requirement | SFR - 000 | XX |
| 성능 - PER | Performance Requirement | PER - 000 | XX |
| 시스템 장비 구성 - ECR | Equipment Composition Requirement | ECR - 000 | XX |
| 인터페이스 - SIR | System Interface Requirement | SIR - 000 | XX |
| 데이터 - DAR | DAR (Data Requirement) / CNR (Consulting Requirement) | DAR - 000 | XX |
| 테스트 - TER | TER (Test Requirement) | TER - 000 | XX |
| 보안 - SER | SER (Security Requirement) | SER - 000 | XX |
| 품질 - QUR | QUR (Quality Requirement) | QUR - 000 | XX |
| 제약 사항 - COR | COR (Constraints Requirement) | COR - 000 | XX |
| 프로젝트 관리 - PMR | PMR (Pjt Management Requirement) | PMR - 000 | XX |
| 프로젝트 지원 - PSR | PSR (Pjt Support Requirement) | PSR - 000 | XX |
| 유지·관리 수행 - MPR | Maintenance Project Requirement | MPR - 000 | XX |
| 유지·관리 인력 - MHR | Maintenance Human Requirement | MHR - 000 | XX |
| 컨설팅 - CNR | Consulting Requirement | CNR - 000 | XX |
| 공사 - ENR | Engineering Requirement | ENR - 000 | XX |
| 합계 | | | XXX |



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

- 근태 관리 시스템의 사용 사례 다이어그램(예)
- 요구사항 목록

| 요구사항 분류 | 요구사항 고유 번호 | 요구사항 명칭 |
|---|------------|--------------|
| 분석/설계 - APR (Analysis Plan Requirement) | APR-001 | 요구사항 수집 및 확정 |
| 분석/설계 - APR | APR-002 | 프로세스 분석 |
| 분석/설계 - APR | APR-003 | 화면 및 DB 설계 |
| 기능 - SFR (System Function Requirement) | SFR-001 | 출퇴근 현황 조회 |
| 기능 - SFR | SFR-002 | 관리자 기능 |
| 기능 - SFR | SFR-003 | 외출 신청 기능 |
| 기능 - SFR | SFR-004 | 유형 근무제 관리 기능 |
| 기능 - SFR | SFR-005 | 특근 신청 관리 기능 |
| 기능 - SFR | SFR-006 | 최종 점검 관리 기능 |
| 기능 - SFR | SFR-007 | 초과 근무 관리 기능 |
| 기능 - SFR | SFR-008 | 권한 관리 기능 |



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

- 근태 관리 시스템의 사용 사례 다이어그램(예)
- 요구사항 목록

| 요구사항 분류 | 요구사항 고유 번호 | 요구사항 명칭 |
|--|------------|--------------|
| 성능 - PER (Performance Requirement) | PER-001 | 기존 시스템 성능 유지 |
| 인터페이스 - SIR (System Interface Requirement) | SIR-001 | 시스템 연계 |
| 인터페이스 - SIR | SIR-002 | 직관적 UI |
| 테스트 - TER (Test Requirement) | TER-001 | 단위 테스트 |
| 테스트 - TER | TER-002 | 통합 테스트 |
| 보안 - SER (Security Requirement) | SER-001 | 보안 기준 준수 |
| 보안 - SER | SER-002 | 응용 및 DB 보안 |
| 보안 - SER | SER-003 | 계정 및 권한 설정 |
| 품질 - QUR (Quality Requirement) | QUR-001 | 기능 구현의 정확성 |
| 품질 - QUR | QUR-002 | 상호 운용성 |



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

- 근태 관리 시스템의 사용 사례 다이어그램(예)
- 단위 기능 요구 사항 명세서

| 요구사항 분류 | 요구사항 고유 번호 | 요구사항 명칭 |
|--|------------|--------------|
| 기능 - SFR (System Function Requirement) | SFR-001 | 출퇴근 현황 조회 |
| | | |
| | | |
| 기능 - SFR | SFR-002 | 관리자 기능 |
| 기능 - SFR | SFR-003 | 외출 신청 기능 |
| 기능 - SFR | SFR-004 | 유연 근무제 관리 기능 |
| 기능 - SFR | SFR-005 | 특근 신청 관리 기능 |
| 기능 - SFR | SFR-006 | 최종 점검 관리 기능 |
| 기능 - SFR | SFR-007 | 초과 근무 관리 기능 |
| 기능 - SFR | SFR-008 | 권한 관리 기능 |

| 구분 | 내용 |
|------------|-----------|
| 요구사항 고유 번호 | SFR-001 |
| 요구사항 명칭 | 출퇴근 현황 조회 |
| 요구사항 분류 | 근태 현황 관리 |
| 필수 여부 | 필수 |
| 정의 | 출퇴근 현황 조회 |

| | |
|------------|----------|
| 요구사항 고유 번호 | SFR-002 |
| 요구사항 명칭 | 관리자 기능 |
| 요구사항 분류 | 근태 현황 관리 |
| 필수 여부 | 필수 |
| 정의 | 관리자 기능 |



2. “소프트웨어 구현(Implementation)”이란?

“구현”을 위해 알아야 하는 핵심 요소

② 설계 원칙을 실제 코드로 녹여내는 기술
개발은 설계를 “그냥 배움”이라면
구현은 설계를 “실제로 코드에 녹여내는 능력”입니다.

● 필수 설계 스킬

- SOLID 원칙 (SRP, OCP, LSP, ISP, DIP)
- 의존성 주입(DI)
- 인터페이스 기반 개발
- 아키텍처
- 모듈 분리
- 계층 구조(Layered Architecture)
- 클린 아키텍처 적용
- DTO / Domain / Service / Infra 클래스 구분



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

사용자 스토리 (User Story)

•Agile 포맷: "As a <role>, I want to <feature>, so that <benefit>."

o예: "나는 <배달 앱 사용자>로서, <이전 주문 내역을 보고> 싶다. 왜냐하면 <자주 먹는 메뉴를 빠르게 재주문하기 위해서>이다."

INVEST 원칙: Independent(독립적), Negotiable(협상 가능), Valuable(가치 있는), Estimable(추정 가능), Small(작은), Testable(테스트 가능).

사용자 스토리 구체화

1. 누가(Who): 어떤 역할의 사용자인가?
2. 무엇을(What): 어떤 기능이 필요한가?
3. 왜(Why): 비즈니스 가치는 무엇인가?



2. “소프트웨어 구현(Implementation)”이란?

“구현”을 위해 알아야 하는 핵심 요소

1. 회원가입 (Sign Up)

US-001 일반 사용자 회원가입

나는 일반 사용자로서, 회사를 선택하고 내 정보를 입력하여 계정을 만들고 싶다.

그래야 나의 권한에 맞는 서비스를 사용할 수 있다.

US-002 보안 사용자 회원가입 (2FA)

나는 민감 정보 접근이 가능한 사용자로서, 보안 강화를 위해 2단계 인증 (2FA)을 설정하고 싶다.

그래야 계정 도용을 방지할 수 있다.

US-003 관리자의 사용자 승인

나는 관리자로서, 새로 가입 요청된 사용자 계정을 확인하고 승인/거절할 수 있어야 한다.

그래야 조직 보안 기준을 유지할 수 있다.



2. “소프트웨어 구현(Implementation)”이란?

“구현”을 위해 알아야 하는 핵심 요소

2. 로그인 (Log In)

US-004 사용자 로그인

나는 등록된 이메일과 비밀번호로 로그인하고 싶다.

그래야 내 계정 정보를 바탕으로 서비스를 이용할 수 있다.

US-005 자동 로그인 유지

나는 자주 로그인하지 않도록 '자동 로그인 유지' 기능을 사용하고 싶다.

그래야 반복적으로 로그인할 필요 없이 빠르게 접근할 수 있다.

US-006 2단계 인증 로그인

나는 내 계정 보호를 위해 2단계 인증을 통해 로그인하고 싶다.

그래야 다른 사람이 내 계정에 접근하지 못하게 할 수 있다.

US-007 비밀번호 재설정

나는 비밀번호를 잊었을 때 이메일을 통해 비밀번호를 재설정하고 싶다.

그래야 다시 시스템에 접근할 수 있다.



2. “소프트웨어 구현(Implementation)”이란?

"구현"을 위해 알아야 하는 핵심 요소 시나리오 작성 요소

1. 정상 흐름: 성공적인 실행 경로
2. 대안 흐름: 다른 선택지가 있는 경우
3. 예외 흐름: 오류 상황 처리

| | |
|---------|------------------------------|
| 시나리오 ID | UC-ORDER-001 |
| 시나리오 명 | 상품 주문 및 결제 |
| 액터 | 고객, 결제 시스템 |
| 목표 | 선택한 상품을 주문하고 결제를 완료한다 |
| 전제조건 | 고객이 로그인되어 있고, 장바구니에 상품이 담겨있음 |



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

시나리오 1: 기본 회원가입 플로우

사용자는 '회원가입' 버튼 클릭

회사 이메일, 비밀번호, 이름, 부서 입력

'약관 동의' 체크 후 제출

관리자 승인 대기 상태로 전환

관리자는 관리자 대시보드에서 승인

사용자에게 승인 완료 이메일 전송



2. "소프트웨어 구현(Implementation)"이란?

| "구현"을 위해 알아야 하는 핵심 요소

시나리오 2: 2단계 인증 회원가입

사용자는 보안 등급 '높음' 선택

기본 정보 입력 후, OTP 앱(QR 코드) 연동 요청

OTP 코드 확인 → 2차 인증 완료

관리자 승인 필요 시 이메일 발송



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

시나리오 3: 로그인 성공

사용자는 로그인 페이지에서 이메일과 비밀번호 입력

시스템은 비밀번호 해시 확인 후 세션 발급

2FA 사용자는 OTP 입력 후 인증

대시보드로 이동



2. "소프트웨어 구현(Implementation)"이란?

| "구현"을 위해 알아야 하는 핵심 요소

시나리오 4: 로그인 실패 (비밀번호 오류)

사용자가 잘못된 비밀번호 입력

시스템은 로그인 실패 메시지 제공

5회 연속 실패 시 계정 일시 잠금



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

시나리오 5: 비밀번호 재설정

사용자가 '비밀번호를 잊으셨나요?' 클릭

이메일 주소 입력 후 인증 링크 수신

링크 클릭 → 새 비밀번호 입력

비밀번호 변경 성공 후 로그인 가능



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

페르소나(Persona) 설정

- 개념: 우리 서비스의 가장 대표적인 가상의 사용자.
- 구성 요소: 이름, 직업, 나이, 목표(Goals), 좌절 요소(Frustrations), 기술 숙련도.
- Tip: "20대 여성"처럼 포괄적인 인구통계학적 정의보다는, "매일 아침 10분 만에 화장을 끝내고 싶은 신입사원 김미영 씨"처럼 구체적인 상황이 좋습니다.



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

사용자 여정 지도 (User Journey Map)

●개념: 사용자가 서비스를 처음 접하고 목표를 달성하기까지의 시간적 흐름.

●단계별 분석:

1. 행동(Action): 사용자가 무엇을 하는가? (예: 앱 실행 -> 검색 -> 결제)

2터치포인트(Touchpoint): 사용자가 만나는 화면이나 기기.

3. 감정(Emotion): 각 단계에서 사용자가 느끼는 기분 (불안, 기쁨, 지루함).

4. 기회(Opportunity): 부정적 감정을 긍정적으로 바꿀 개선점.



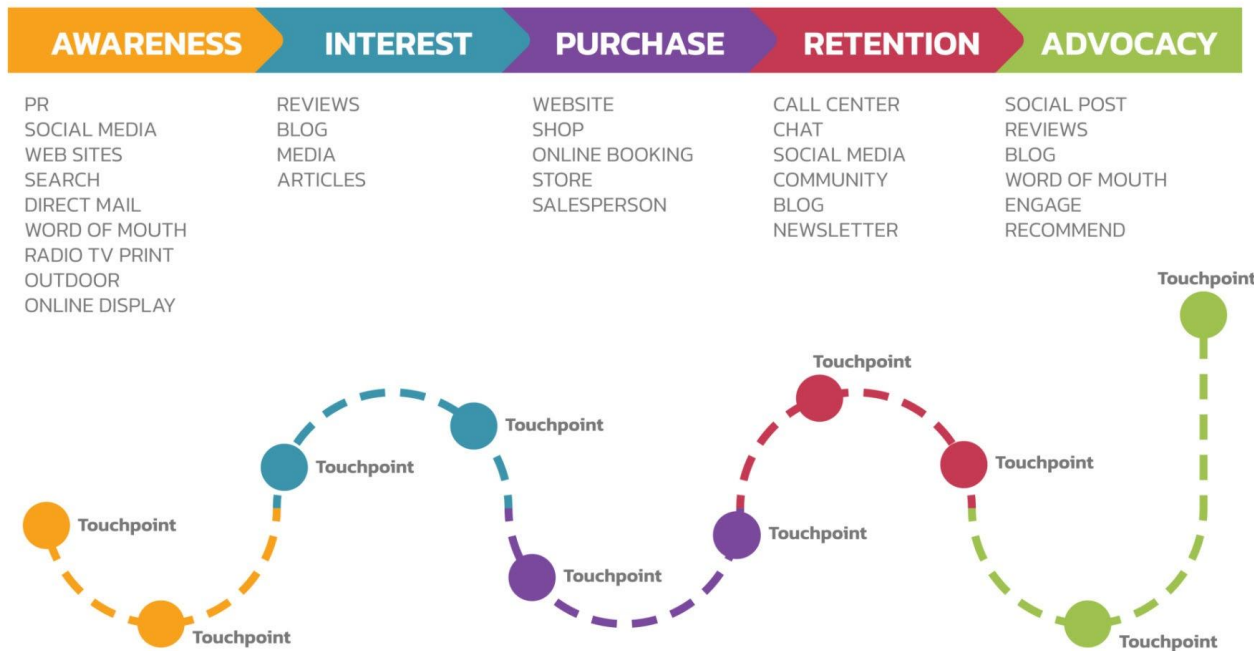
2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소
사용자 여정 지도 (User Journey Map)

●

BUSINESS INFOGRAPHIC

CUSTOMER JOURNEY MAP



2. "소프트웨어 구현(Implementation)"이란?

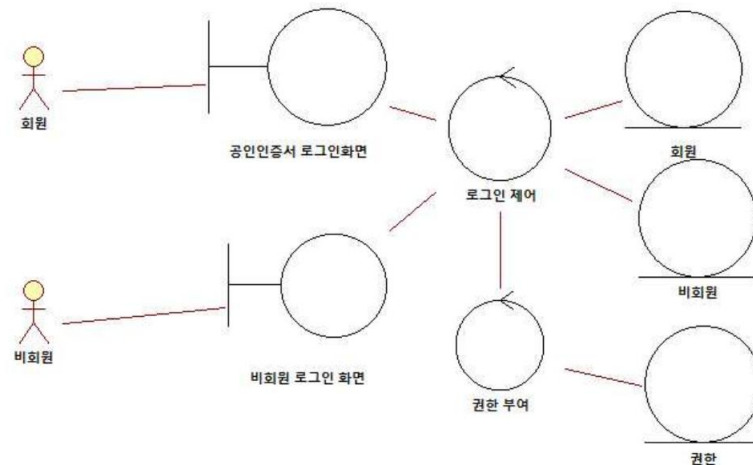
"구현"을 위해 알아야 하는 핵심 요소

런던 스쿨(London School)의 분석 모델 (Analysis Model)
ECB(Entity-Control-Boundary) 패턴

분석 클래스의 스테레오 타입

| 역할구분 | 스테레오 타입 | 내용 |
|----------------|--|------------------------------------|
| 경계
Boundary | <<boundary>>
 | 시스템과 외부 액터와의 상호작용을 담당하는 클래스 |
| 엔터티
Entity | <<entity>>
 | 시스템이 유지해야 하는 정보를 관리하는 기능을 전담하는 클래스 |
| 제어
Control | <<control>>
 | 시스템이 제공하는 기능의 로직 및 제어를 담당하는 클래스 |

분석 클래스 다이어그램 예시





2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소 핵심 개념 정리 (핵심 3종 세트)

| 분석 요소 | 스테레오타입 | 의미 (런던스쿨 기준) | 실제 구현 시 고려사항 |
|----------|--------------|---|--|
| Boundary | <<boundary>> | 시스템과 외부 액터(사람, 외부 시스템) 사이의 인터페이스 → UI 화면, API 엔드포인트, 메시지 수신부 등 | 화면 설계, 컨트롤러(Controller), DTO, Request/Response 객체 |
| Entity | <<entity>> | 시스템이 영속적으로 관리해야 하는 업무 핵심 데이터 → 도메인 객체, DB 테이블과 거의 1:1 매핑 | JPA Entity, 도메인 모델, 테이블 설계 |
| Control | <<control>> | Boundary와 Entity 사이에서 오케스트레이션 / 업무 로직 조율을 담당 → Use Case를 실현하는 중심 객체 | Service 클래스, UseCase 클래스, Transaction Script, Domain Service |



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

경계 (Boundary)

- 시스템과 외부 액터(사용자)의 접점 → 이름이 보이지 않지만 일반적으로 다음과 같을 가능성이 높음
- 어떤화면 (예: 주문등록화면, 회원가입화면 등)
- 또는 REST API 컨트롤러 (OrderController 등)

엔티티 (Entity)

- <<entity>> 로 표시된 원 3개
- 주문 (Order)
- 주문항목 (OrderItem)
- 상품 (Product) 또는 결제 (Payment) 등
- → "시스템이 유지해야 하는 정보를 관리하는 가능한 전형적인 클래스"들

제어 (Control)

- <<control>> 하나 → 대부분의 경우 Use Case의 중심 컨트롤 객체
- 예: 주문등록Control, 회원가입Control, 결제Control 등
- 이름 예시: 주문등록Control, PlaceOrderControl, RegisterOrderUseCase 등



2. “소프트웨어 구현(Implementation)”이란? .

“구현”을 위해 알아야 하는 핵심 요소

사용자 스토리 (예상)

“로그인한 고객으로서, 장바구니에 담은 상품들을 주문하고 결제까지 완료하고 싶다.”



2. “소프트웨어 구현(Implementation)”이란?

“구현”을 위해 알아야 하는 핵심 요소

주요 시나리오 (Happy Path)

1. 고객이 [주문하기] 버튼을 누른다 → Boundary (주문등록화면 또는 OrderController)
2. Boundary는 필요한 데이터를 수집해서 <<control>> 주문등록Control 을 호출<-요청 데이터: 고객ID, 배송지 정보, 장바구니 상품 목록, 결제수단 등
3. 주문등록Control 이 트랜잭션 시작
4. Control → <<entity>> 주문 생성
5. Control → <<entity>> 주문항목 들 생성 (장바구니 기반)
6. Control → <<entity>> 상품 재고 차감
7. Control → 외부 결제 게이트웨이 연동 (또 다른 Boundary 호출 가능)
8. 모든 Entity 저장 (persist)
9. Control → Boundary에게 성공 응답 반환
10. Boundary → 사용자에게 “주문이 완료되었습니다” 화면 또는 JSON 응답



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

구현 파일 구조 예시 (Spring Boot 기준)

/boundary

- └─ OrderController (또는 OrderRegisterScreen)
 - @RestController 또는 UI View

/control

- └─ PlaceOrderControl (또는 PlaceOrderService, PlaceOrderUseCase)
 - @Transactional
 - 주문 생성 로직 전체 오케스트레이션

/entity

- └─ Order.java (@Entity)
- └─ OrderItem.java (@Entity)
- └─ Product.java (@Entity)
- └─ Payment.java (@Entity) // 필요시



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

런던 스킴 vs 다른 스타일 비교

| 구분 | 런던 스킴 | 전통 3계층 (Transactional Script) | 헥사고날/클린 아키텍처 |
|--------------|----------------------------------|-------------------------------|--------------------------------|
| Use Case 가시성 | 명확 (Control 클래스 1개 = 1 Use Case) | Service 클래스에 다 섞임 | UseCase 인터페이스 존재 가능 |
| 도메인 로직 위치 | Entity + Domain Service | 주로 Service 계층 | 도메인 계층 (Entity + Service) |
| 트랜잭션 경계 | Control | Service | UseCase 또는 Application Service |
| 코드 읽기 쉬움 | 매우 높음 (분석 모델과 1:1) | 보통 | 높음 |
| 학습 곡선 | 약간 가파름 (스테레오타입 이해 필요) | 낮음 | 중간 |



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

③ 코드 품질과 예외 처리

실제 제품은 예외 처리와 안전성이 80%.

● 구현 시 필수 요소

- try-catch 전략
- 오류 메시지 표준화
- Validation 계층
- 로그 레벨 관리(INFO / WARN / ERROR)
- 실패 대비 로직 (fallback, retry)
- 안전한 리소스 해제(사용 후 close)
- 메모리 누수 관리



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

④ I/O와 외부 연동의 안정적 처리

실제 소프트웨어는 외부 I/O 문제가 대부분입니다.

- DB 연결 시 커넥션 풀 관리
- API 타임아웃
- 재시도 정책(backoff)
- 네트워크 장애 대응
- 파일 입출력 예외 처리
- 비동기 처리(async/await)



2. “소프트웨어 구현(Implementation)”이란?

“구현”을 위해 알아야 하는 핵심 요소

⑤ 테스트 → 구현 → 리팩토링 루프 안정화
반드시 알아야 할 테스트 스킬

- 단위 테스트(Unit Test)
- 통합 테스트(Integration Test)
- Mocking (Mockito, unittest.mock 등)
- Golden Master Test
- API 테스트(Postman, REST Client)
- 부하 테스트(Load Test)

구현의 핵심은

“변경해도 안 깨지는 안정성 확보”.



2. “소프트웨어 구현(Implementation)”이란?

"구현"을 위해 알아야 하는 핵심 요소

⑥ 배포(Deployment)와 운영(Operation)

실제로 구현은 “동작하는 소프트웨어를 배포하는 것”까지 포함합니다.

- 배포에 필요한 지식

- GitHub Action / GitLab CI
- Docker 이미지 만들기
- 환경 변수 관리
- staging / production 구분
- 배포 시 롤백 전략
- 로깅/모니터링 시스템

- 운영(Ops)에 필요한 지식

- CPU/Memory/IO 모니터링
- 로그 분석
- 장애 대응
- API 응답 시간 측정
- 버전 관리
- SLA/SLO 개념



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

⑦ 보안·성능·확장성까지 고려하는 능력

● 보안

- 인증/인가
- JWT
- 입력값 검증(OWASP 기준)
- SQL Injection 방지
- HTTPS / TLS
- 암호화(Hash, Salt)

● 성능

- 캐싱(Redis)
- 페이징 처리
- Lazy Loading
- 쿼리 최적화
- 병렬 처리(Thread, async)
- CDN 활용



2. "소프트웨어 구현(Implementation)"이란?

"구현"을 위해 알아야 하는 핵심 요소

- 확장성
 - 무상태(Stateless) 설계
 - 스케일 아웃 전략
 - 마이크로서비스 개념
 - API Rate-Limit



2. “소프트웨어 구현(Implementation)”이란? .

| 구현할 수 있기 위해서는 ...

1단계: 기능을 완전히 “작동시키는 능력”

- 요구 → 기능 단위로 분해
- 예외 처리 포함
- 단위 테스트 포함
- API 문서 포함
- 로깅 포함
- 오류 메시지 포함
- 기본 배포 파이프라인 구성



2. “소프트웨어 구현(Implementation)”이란? .

| 구현할 수 있기 위해서는 ...

2단계: 유지보수 가능한 코드 만들기

- SOLID 적용
- 설계 패턴 적용
- 모듈 분리
- Repository / Service / Controller 패턴
- Interface+DI 기반 구조 적용



2. “소프트웨어 구현(Implementation)”이란? .

| 구현할 수 있기 위해서는 ...

3단계: 운영 가능한 제품 만들기

- Docker 컨테이너화
- 환경 변수(.env) 관리
- 자동 배포
- 로그 모니터링
- 장애 대응 플로우 정립



3. UI/UX 관련 추가 지식

가치

- 즐거움, 사용자 관점에서의 좋은 경험
- 형태와 색은 디자인의 가치를 생산하는 근원. 디자인이 제공하는 가치는 다음과 같습니다.
 1. 기능성 가치: 제품이나 서비스의 기능을 효과적으로 수행할 수 있도록 디자인됩니다.
 2. 사용자 경험 가치: 제품이나 서비스를 사용하는 사용자들이 만족할 수 있는 사용자 경험을 제공합니다.
 3. 브랜드 가치: 제품이나 서비스의 디자인이 브랜드 이미지와 일치하고 브랜드 가치를 제고하는 역할을 합니다.
 4. 경제적 가치: 제품이나 서비스의 디자인이 경제적인 효과를 가져오며, 기업의 수익과 경쟁력을 향상시킵니다.
 5. 환경 가치: 제품이나 서비스의 디자인이 환경을 고려하고, 지속 가능한 방식으로 디자인되어야 합니다.
- 서양 철학자 존 로크
 - 1차 속성 : 형태
 - 2차 속성 : 색



3. UI/UX 관련 추가 지식

조형능력

- 조형능력은 시각적인 형태와 구성 요소를 이해하고, 이를 조합하여 창조적인 작품을 만들어내는 능력을 의미.
- 사람들의 시선을 사로잡을 멋진 디자인을 만들기 위해서는 조형능력이 뛰어나야 한다.
- 수렴
- 귀
 - 음정
 - 박자
 - 멜로디
- 눈
 - 형태
 - 색



3. UI/UX 관련 추가 지식

형태

- 점

- 눈으로 볼 수 있는 가장 작은 형태이며, 이론 상으로 존재하는 가장 기본이 되는 요소이다. 모든 형태의 근원이자 출발점이라 할 수 있다. 연속적인
- 점은 입체감, 방향감, 동체를 느끼게 해준다. 점이 움직이면 선, 많은 점이 모이면 면의 형태를 표현한다. 점의 크기와 배치상의 조밀도의 따라 면에
- 서의 요철느낌을 표현한다. 크기를 갖지 않고 위치만을 가지고 있는 것이 점이다.



3. UI/UX 관련 추가 지식

형태

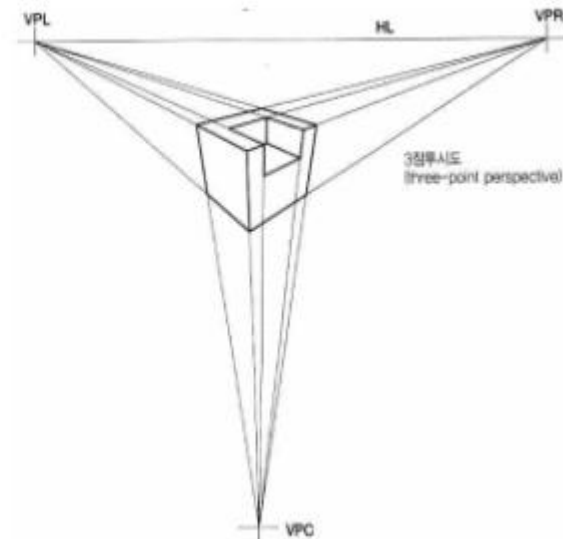
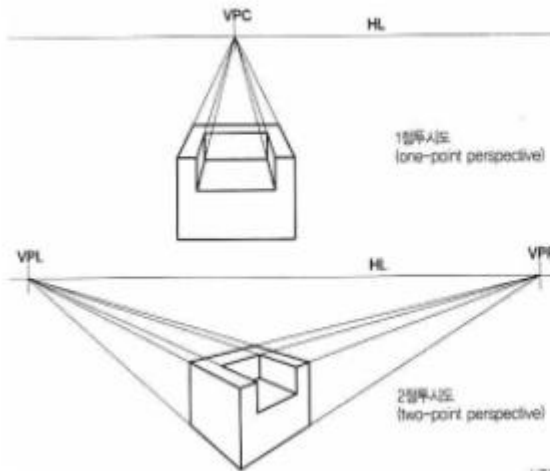
● 선

- 두 개 이상의 점이 연결된 것으로 모든 조형요소의 기초이다. 점이 모여서 이루어지는 선은 사물의 윤곽을 나타내기도 하고 공간을 구분하며 때로는 덩어리를 암시하기도 한다.
- 모든 그림은 선에서 시작된다고 할만큼 그림을 그린다는 행위는 곧 선의 사용을 의미한다. 손만으로도 예술적 의미를 갖는 개성적인 그림을 그릴 수 있다. 선은 길이만 있고 굵기와 넓이 부피가 없는 것이다. 그러나 우리는 선 없는 그림을 생각할 수 없을 만큼 선은 그림에서 매우 중요한 요소가 된다. 화면에 한줄기의 선을 그으면 공간은 둘이 된다. 선은 물체의 윤곽을 나타내는 동시에 선 그 자체의 독자적인 감정을 가지고 있다. 동양화는 선이 생명인 동시에 선이 예술이다. 서양화에서는 면이 강조되었으나 근대회화에서는 고흐,마티스 등은 선을 잘 살려 그리고 있다

3. UI/UX 관련 추가 지식

형태

- 면
 - 연속적인 선에 의하여 표현되는 것을 면이라 한다. 면은 면적을 지닌 2차원의 평면으로 사물의 일반적인 테두리를 이룬다. 면에 의하여 선과 형태가 형성되며 면이 입체화 되면 덩어리가 나타난다.
- 공간
- 색
- 투시도 종류





3. UI/UX 관련 추가 지식

디자인의 시각적 인식 과정

- 복잡한 시내 한복판을 지나가는 경우를 생각해 볼 때,
- 대부분의 사물들은 그냥 스쳐 지나가지만 눈에 띄는 외모, 화려한 건물 등은 한눈에 들어 옴
- 반면, 형태가 혼란스럽거나 아무 내용도 없는 대상들은 눈에 안 들어 옴
- 보는 작용과 이해하는 작용이 융합
 - 감각-혼란 최소화, 윤곽의 명확성은 최대화
 - 지각-시각적 인상 최대화, 비례 최대화, 색의 조화 최대화
 - 미적 판단-눈에 보이는 형상에 국한되지 않고 철학이나 인문학, 취미 수준과 같은 다양한 가치들이 개입



3. UI/UX 관련 추가 지식

디자인의 시각적 인식 과정

- 디자인의 시각적 인식 과정은 인간의 눈과 뇌가 구성 요소들의 배치와 물리적 속성(색, 크기 등)에 근거하여 전체 패턴을 파악하는 시지각 원리를 근거로 이루어진다.
- 이 시지각 이론은 1900년대 초 독일의 심리학파인 게슈탈트(Gestalt) 학파에 의해 활발하게 연구되었으며, 시지각의 게슈탈트 원리(The Gestalt Principles of Visual Perception)로 정립되었다. 프랑스 지도제작자인 자크 베르탱(Jacques Bertin)은 정보 디자인의 시각화 방법론의 선구자로서, 게슈탈트 이론에 근거해 정보 표현을 위한 그래픽 요소들의 시각화 방법론을 정립하였다.



3. UI/UX 관련 추가 지식

디자인의 시각적 인식 과정

- 베르탱의 저서인 『도표 기호학(semilogoc graphique, 1967)』은 모양, 색, 패턴을 통해 데이터를 표현하고 해석하여 인지하는 이론적 근거를 제시하였고, 데이터 시각화 분야의 근간이 되고 있다. 베르탱 외에도 존 튜키(John W Tukey), 윌리엄 클리블랜드(William Cleveland), 조크 맥킨레이(Jock McKinlay) 등의 영향력 있는 연구자들이 수년에 걸쳐 효과적인 정보 시각화 방법과 이론들을 정립해나가고 있다.



3. UI/UX 관련 추가 지식

1. 게슈탈트 원리

- 뇌가 색과 형태의 조각들을 하나의 패턴으로 보는 원리로서, 게슈탈트는 모양이나 도형을 의미하는 독일어이다.

- 유사성의 법칙(Low of Similarity)

유사한 요소(형태, 색, 크기, 밝기)끼리 그룹 지어 하나의 패턴으로 보이게 한다.

- 근접성의 법칙(Low of Proximity)

시공간적으로 서로 가까이 있는 것들을 함께 집단화하여 본다.

- 단순성의 법칙(Law of Simplicity)

주어진 조건으로 최대한 가장 단순한 쪽으로 인식한다.

- 연속성의 법칙(Law of Continuity)

요소들이 부드러운 연속을 따라 함께 묶여 지각된다.

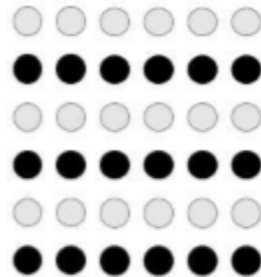
- 폐쇄성의 법칙(Law of Closure)

기존의 지식을 토대로 완성되지 않은 형태를 완성해 인지한다.

3. UI/UX 관련 추가 지식

1. 게슈탈트 원리

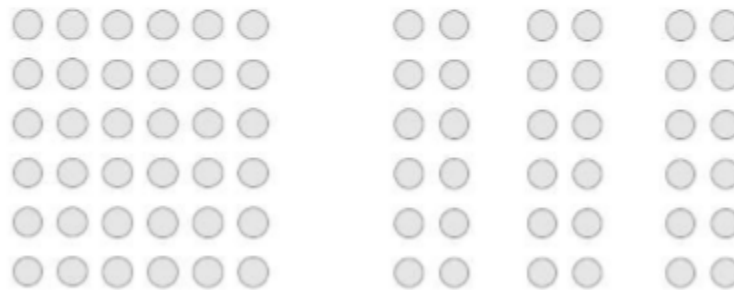
- 뇌가 색과 형태의 조각들을 하나의 패턴으로 보는 원리로서, 게슈탈트는 모양이나 도형을 의미하는 독일어이다.



[그림 3-1] 유사성의 법칙



[그림 3-2] 폐쇄성의 법칙



[그림 3-3] 근접성의 법칙



3. UI/UX 관련 추가 지식

2. 자크 베르탱의 그래픽 7요소

- 자크 베르탱의 그래픽 7요소는 데이터나 정보들을 점, 선, 면, 영역, 입체 등의 기하적 요소인 시각기호로 그리고 이 시각기호를 색, 크기, 위치 등으로 장식함으로써 양적, 질적 데이터의 비교, 관련성, 순서성 등을 인지할 수 있게 하는 시각적 표현 방법을 말한다. 여기서 시각적 표현 방법은 시각기호를 표현하기 위해 사용되는 시각변수로도 부르는데, 그래픽 7요소는 시각변수 7가지 크기(size), 모양(shape), 밝기(lightness/value), 색(color), 방향(orientation), 질감(texture), 위치(location)를 말한다.



3. UI/UX 관련 추가 지식

2. 자크 베르탱의 그래픽 7요소

(1) 크기(size)

- 선, 면, 원 등 동일한 기하적 요소들이 같은 크기를 가진 경우 상대적으로 하나만 작게 표시하면 작게 표현한 것만이 눈에 띄게 된다. 이는 크기가 크거나 작은 것보다는 같이 있는 요소들의 관계에 의해 구별되기 때문이다. 또 클수록 시선이 주목된다. 그러므로 크기는 데이터의 수치와 대응하여 표현할 수 있으며 데이터의 크기 순서로도 구분이 가능한 표현 방법이 된다. 따라서 양과 중요도를 인지하는 시각 변수로 사용할 수 있다.



3. UI/UX 관련 추가 지식

2. 자크 베르탱의 그래픽 7요소

(2) 모양(shape)

- 같은 모양 중 하나만 다른 모양이 있는 경우 다른 모양이 강조되어 눈에 띈다. 비슷한 형태 보다는 다른 형태로 주어지면 강조돼 보인다. 인간의 눈은 외곽선을 보고 대상을 인지하게 되는데, 형태를 구분하는 능력은 색이나 크기를 인지하는 능력보다 고도의 기술이므로, 모양만으로 큰 대비 효과를 기대할 수는 없다.



3. UI/UX 관련 추가 지식

2. 자크 베르탱의 그래픽 7요소

(3) 색(color)

- 색은 인간의 사물을 인지하는 과정에서 깊이 단서가 되어 전경과 배경을 구분하는 주요한 요인이 되므로 주의 깊게 사용해야 하며, 동일한 색에 대해서도 문화적 환경에 따라 다른 의미로 인지할 수 있는 요소이기도 하다. 같은 색을 이용한 것 중에서 다른 색을 사용한 요소가 더 잘 보인다. 색을 이용한 강조를 위해 보색을 사용하는데, 인쇄물의 보색과 스크린 상의 보색이 다르므로 이를 이해하고 사용해야 한다. 색은 본질적으로 정량적인 속성이 아니므로 순서를 매기기에 적합하지 않다.



3. UI/UX 관련 추가 지식

2. 자크 베르탱의 그래픽 7요소

(4) 밝기(명도 lightness/value)

- 동일한 색에 대하여 동일한 수준의 밝기(명도)를 가진 요소 중 일부만 다른 밝기를 줄 때 눈에 띄게 된다. 밝기의 높고 낮음은 색보다 오히려 명시성에 영향을 준다.

(5) 방향(orientation)

- 하나만 다른 방향으로 기울이면 방향이 다른 요소에 시선이 끌리게 된다. 이는 동일한 방향을 갖는 요소가 반복될 경우 시선이 이를 벗어난 변화를 감지하기 때문이다.



3. UI/UX 관련 추가 지식

2. 자크 베르탱의 그래픽 7요소

(6) 질감(texture)

- 같은 색상과 모양이라도 질감을 다르게 준 것만 강조된다. 질감을 지나치게 사용할 경우 오히려 정보를 인지하는 데 방해가 될 수 있으므로 주의하여 사용해야 한다.

(7) 위치(location)

- 위치를 다르게 함으로써 강조할 수 있다. 특정한 요소의 상대적인 위치를 확인하여 주변의 다른 요소와의 관계를 비교하게 되는데, 크기와 같이 위치는 수치로 변환할 수 있다. 정보의 상하 구조나 대소 비교를 효과적으로 전달할 수 있다.



3. UI/UX 관련 추가 지식

조형 감각을 키우기 위한 방법론

- 경험
 - 조형 감각은 이론이 아니라 경험의 축적
- 조형적 대상
 - 산/강/노을
 - 박물관의 고대 유물
 - 주변에서 흔히 볼 수 있는 사물
- 조형 공부의 중심
 - 형태와 색으로 만들어진 디자인은 최종적으로 사람들에게 받아들여져야 한다.
- 사람
 - 디자인을 대하는 사람의 존재성을 고려해야 하는 것으로 조형 공부한 조형 이론을 공부하는 것이 아니라, 디자인을 보는 사람의 시각적 인식과정에 관한 공부
- 주변 사물들에 녹아 있는 조형성의 체험
 - 좋은 안목은 아주 사소한 사물이나 현상들에서부터 수준 높은 예술 작품에 이르기까지 세밀하게 관찰하는 노력을 통해서 만들어진다.

3. UI/UX 관련 추가 지식

조형적 균형

- 조형요소와 비례
 - 디자인은 화면에 들어가는 조형요소나 비례가 잘 조율되어 있다.
- 가시성
 - 배경과 분리된 가시 상태.



있는 정도 또는



3. UI/UX 관련 추가 지식

조형적 균형

- 그룹핑

공간을 이용한 그룹핑

- 공통 기능을 가진 것들을 묶어 일관성을 유지하는 기법.

외곽선을 이용한 그룹핑

- 외곽선을 이용한 그룹핑은 모여 있는 정보 주변에 외곽선을 넣어, 비슷한 내용끼리는 한 덩어리로 보이게 하는 방법.

컬러를 이용한 그룹핑

- 컬러를 이용한 그룹핑은 관련 있는 내용에 동일한 컬러를 사용하는 방법.

- 그리드 시스템

- 공간을 효율적으로 활용하기 위해 일정한 구역(Grid)으로 나눠 놓은 것을 말한다.

- 강조성

- 불필요한 요소를 과감히 없애 중요한 정보를 다른 정보와 차별화시키는 작업을 말한다.



3. UI/UX 관련 추가 지식

조형적 균형

- 반전 기법(구성 요소가 선택된 경우)
 - 구성 요소에 마우스 포인터가 닿으면 이미지가 반전되도록 하는 강조 기법을 사용.
- 컬러 대비 기법(중요한 글자 강조)
 - 중요한 문구의 컬러를 나머지 문구와 대비시켜 강조하는 기법을 사용.
- 글자의 굵기 차, 밑줄, 이탤릭체를 이용한 기법
 - 꼭 필요한 구성 요소에만 사용한다. 중요한 문구나 단어에 글자 효과를 넣어 강조하는 기법.
- 아이콘이나 그래픽을 이용하는 방법
 - 중요한 콘텐츠는 아이콘을 넣어 강조하는 기법.
- 글자의 강약을 대조



3. UI/UX 관련 추가 지식

조형적 균형

● 일관성

- 하나의 웹 사이트 또는 여러 웹 사이트에서 지켜지고 있어 굳이 학습하지 않아도 쉽게 사용할 수 있는 법칙을 말한다.

내부적 일관성

- 일정한 규칙에 따라 웹 사이트를 제작하여 일관성이 느껴지도록 디자인하는 것.
- 기능을 하는 버튼의 디자인 일관성

글자와 그래픽의 일관성

- 일관된 레이아웃
- 조절 요소의 위치, 크기, 조작법 등의 통일된 약속
- 레이블링, 버튼 위치, 일관성 있는 시각적 요소들의 통일된 구성

외부적 일관성

- 사용자가 여러 프로그램 사이에서 일관성을 느낄 수 있도록 디자인하는 것.
- 게시판의 기본적인 아이콘의 일관성
- Home으로 이동하는 내비게이션의 일관성
- 웹 사이트 하단의 푸터 영역에 있는 저작권 부분의 일관성

3. UI/UX 관련 추가 지식

정보 디자인 요소

- 정보를 구성하여 효율적으로 사용할 수 있도록 하는 디자인 기술 및 업무를 말한다. 이러한 디자인은 복잡하거나 구조화되지 않은 데이터를 시각적으로 표현하여 그 뜻을 명확하고 분명하게 보이게 한다.
- 내비게이션 디자인
 - 사이버 공간에서 사용자가 쉽게 이동할 수 있도록 수단을 디자인하는 것이다.
- 폼 디자인
 - 회원 정보, 결제 정보, 배송 정보 등의 입력 양식을 디자인하는 것이다.
 - 폼 디자인의 예

(a) 변경 전 : 정보의 양이 많아 복잡해 보임

(b) 변경 후 : 관련 있는 정보를 그룹으로 묶어 정리되어 보임

3. UI/UX 관련 추가 지식

정보 디자인 요소

- 아이콘 레이블
 - 아이콘 이미지와 명칭을 함께 쓰는 레이블이다. 글자 레이블보다 많은 정보를 전달할 수 있지만 몇 가지 제약도 따른다.



아이콘 예제입니다(해당 클래스 이름으로 사용)

EXAMPLE

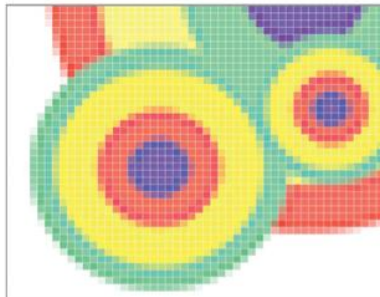
✉ 📀 📁

```
<span class="glyphicon glyphicon-envelope" aria-hidden="true"></span>
<span class="glyphicon glyphicon-floppy-disk" aria-hidden="true"></span>
<span class="glyphicon glyphicon-folder-open" aria-hidden="true"></span>
```

3. UI/UX 관련 추가 지식

기술 디자인 요소

- 디자인을 구현하는 데 필요한 기술을 습득하여 기술을 반영한 디자인을 말한다.
- 기술 디자인 요소 : 로딩 속도와 상호작용
 - 디자인을 할 때는 여전히 빠른 로딩 속도를 중요하게 고려해야 한다. 문제는 웹 페이지 로딩 속도와 디자인 효과가 상반된 관계에 있다는 점이다.
- 비트맵 이미지와 벡터 이미지
 - 비트맵 이미지 : 한 가지의 컬러 정보를 가진 픽셀을 모아 만든 오브젝트를 말한다.
 - 벡터 이미지 : 기준점과 방향선 등의 정보를 모아 만든 오브젝트를 말한다.



비트맵

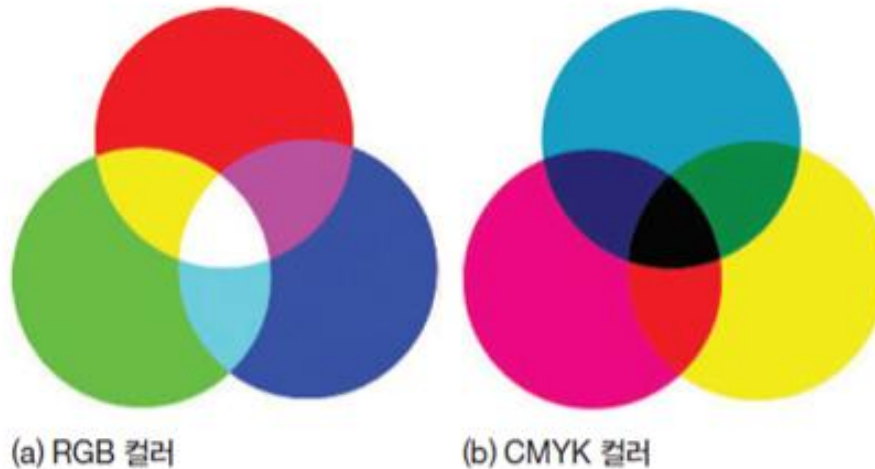


벡터

3. UI/UX 관련 추가 지식

기술 디자인 요소

- 파일 형식과 컬러 모드
- 파일 형식 : 대표적인 이미지 파일 형식에는 GIF, JPEG, PNG가 있다.
- 컬러 모드 : 대표적인 컴퓨터 그래픽 컬러 모드에는 RGB, CMYK, Index 모드가 있다.





3. UI/UX 관련 추가 지식

컬러별 특징

- 빨강
 - 활력, 에너지, 열정, 따뜻함, 사랑, 유혹 같은 긍정적인 상징과 피, 위험, 범죄, 노여움, 공격성, 복수, 재난 같은 부정적인 상징이 공존한다.
- 주황
 - 온화함, 따뜻함, 인자함, 에너지, 활동적, 균형적 같은 긍정적인 상징과 피상적, 불안, 경계, 증오, 변덕 같은 부정적인 상징이 공존한다
- 노랑
 - 희망, 낙천적, 행복, 즐거움, 생동감 같은 긍정적인 상징과 배신, 질투, 경고, 비겁함 같은 부정적인 상징이 공존한다.
- 초록
 - 건강, 자연, 젊음, 생명, 신선함, 활력, 보안성 같은 긍정적인 상징과 미숙함, 다듬어지지 않음, 엄격함, 부패, 부식 같은 부정적인 상징이 공존한다.
- 파랑
 - 신뢰감, 성공, 충성심, 보수적, 안전, 첨단, 젊음, 청결, 노동력, 유용함, 강인함 같은 긍정적인 상징과 우울함, 슬픔, 천박함, 황무지, 황폐화 같은 부정적인 상징이 공존한다.



3. UI/UX 관련 추가 지식

컬러별 특징

- **남색**
 - 카리스마, 신뢰, 비즈니스, 전문성, 안정, 완벽함, 무한함, 진지 같은 긍정적인 상징과 무거움, 차가움, 냉정, 의심, 긴장 등 부정적인 상징이 공존한다.
- **보라**
 - 신비로움, 고상함, 섬세함, 매력적, 전통적, 낭만, 환상 같은 긍정적인 상징과 오만함, 외로움, 불안정, 불행, 죽음, 고독 등 부정적인 상징이 공존한다.
- **분홍**
 - 낭만, 열정, 사랑, 환희, 발랄함, 쾌활함, 감성적, 따뜻함 등 긍정적인 상징과 성적인, 변덕스러운 등 부정적인 상징이 공존한다.
- **갈색**
 - 격조, 품격, 신뢰감, 흙, 편안함, 중후함, 우아함, 전통, 수확 같은 긍정적인 상징과 감퇴, 죽음, 부패, 부식, 지저분함 같은 부정적인 상징이 공존한다.
- **하양**
 - 순수, 청결, 간결, 정직, 결백, 여백, 청렴, 고요, 세련, 우아, 평화 같은 긍정적인 상징과 단조로움, 무기력 같은 부정적인 상징이 공존한다.

3. UI/UX 관련 추가 지식

시각 디자인 요소

- 타깃 사용자가 선호하며 타깃 사용자의 감성을 자극하는 디자인을 제공하면 많은 사람들이 찾는 웹 사이트가 될 것이다. 또한 감성에 맞는 디자인 스타일과 함께 가독성, 여백, 비례 역시 웹 디자인의 중요한 시각 디자인 요소이므로 신경 써서 작업해야 한다.
- 파노라마 배경 이미지 파노라마 배경 이미지
- 사방에서 본 것 같이 크고 긴 배경 사진 이미지
- 시네마 그래프
- 고속으로 연속 촬영한 사진을 붙이거나 영상을 찍어 GIF로 만든 이미지
- 이미지와 글자 혼합 형식
- 사용할 이미지가 잘 보이는지, 글자가 잘 읽히는지, 전달하려는 메시지가 잘 전달되는지 세 가지를 모두 고려하여 디자인해야 한다.
- 이미지와 글자 혼합 형식 예





3. UI/UX 관련 추가 지식

타이포그래피

- 타이포그래피(Typography)란 글자와 글자가 위치한 공간이나 레이아웃 등 글자로 디자인된 상태를 말한다.
- 웹페이지에 사용하는 타이포그래피는 크게 두 가지 방법으로 제작된다. 첫 번째는 워드에 글자를 입력하는 것처럼 HTML을 이용하여 글자를 입력하는 방법이다. 두 번째는 포토샵과 같은 그래픽 프로그램을 이용하여 이미지로 제작하는 방법이다.



3. UI/UX 관련 추가 지식

타이포그래피

- 세리프체

- 글자와 기호를 이루는 획의 일부 끝이 돌출된 형태로 한글 서체에서는 명조체가 속한다. 글자 획이 굵지 않아 크기가 작더라도 시각적으로 자연스럽고 편안한 느낌을 주기 때문에 글자 양이 많은 책이나 신문 같은 인쇄물에 널리 쓰인다.

바탕체 궁서체

(a) 한글 세리프체 : 바탕체와 궁서체

Bodoni Times New Roman

(b) 영문 세리프체 : Bodoni와 Times New Roman

3. UI/UX 관련 추가 지식

타이포그래피

- 산세리프체
 - 글자 획의 굵기가 변하지 않고 가로획과 세로획이 거의 일정하며, 획의 끝에 장식이 없는 것을 말한다. 산세리프체는 인상이 강하고 직설적인 느낌을 주므로 제목이나 간판에 널리 사용된다.

굴림체 돋움체

(a) 한글 산세리프체 : 굴림체와 돋움체

Arial Helvetica

(b) 영문 산세리프체 : Arial과 Helvetica

3. UI/UX 관련 추가 지식

타이포그래피

- 글자 크기와 단위
 - 글자 크기를 지정할 때는 Font-size를 사용한다. 글자 크기로 자주 쓰는 단위에는 px와 em이 있는데 W3C에서는 em을 쓰길 권장한다.
 - <https://typescale.com/>

The screenshot displays the Typescale website interface. On the left, there are controls for 'Base Size' (set to 16px), 'Scale' (set to 1.250 - Major Third), 'Google Fonts' (set to Poppins), and 'Weight' (set to 400). Below these is a 'Preview Text' field containing 'A Visual Type Scale'. At the bottom left, there are links for 'Reset All' and 'Save for Later'. The main area shows a vertical list of text samples, each with its size in rem and px (e.g., 3.052rem/48.83px) and the text 'A Visual Type Scale'. The samples are arranged in a descending scale. At the bottom, there is an advertisement for Adobe Creative Cloud for Teams and two buttons: 'Grab the CSS' and 'Edit on CodePen'.



3. UI/UX 관련 추가 지식

타이포그래피

- 자간
- 자간(Tracking)은 글자 간격으로, 글자의 가운데부터 다음 글자의 가운데 까지 거리를 뜻한다.

| | |
|------|---------------|
| 글자간격 | -100으로 설정한 경우 |
| 글자간격 | 0으로 설정한 경우 |
| 글자간격 | 200으로 설정한 경우 |

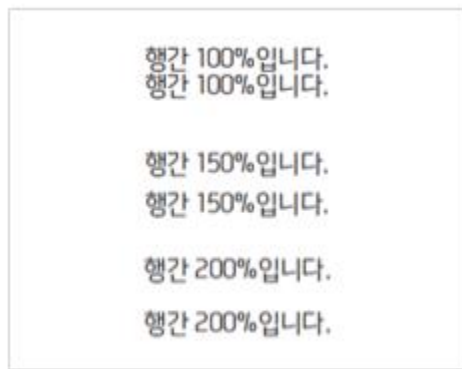
(a) 자간을 -100, 0, 200으로 설정한 예



3. UI/UX 관련 추가 지식

타이포그래피

- 행간
 - 행간행간(Leading)은 줄 간격으로, 글자 아랫부분부터 다음 줄 글자의 아랫부분까지 거리를 뜻한다.



(a) 행간을 100%, 150%, 200%로 설정한 예

3. UI/UX 관련 추가 지식

타이포그래피

- 장평
- 장평(Horizontally Scale)은 글자의 폭과 높이 비율을 말한다.



(a) 장평을 70%, 100%, 150%로 설정한 예



3. UI/UX 관련 추가 지식

네비게이션

● GNB / LNB

네비게이션 예제입니다.

EXAMPLE

정부3.0정보공개

주요사업

고객광장

알림마당

```
<div id="gnb">
  <ul class="lst_gnb">
    <li><a href="#">정부3.0정보공개</a></li>
    <li><a href="#">주요사업</a></li>
    <li><a href="#">고객광장</a></li>
    <li><a href="#">알림마당</a></li>
  </ul>
</div>
<div id="gnb_sub">
  <ul class="lst_gnb_sub">
    <li>
      <ul>
        <li><a href="#">정보공개</a></li>
        <li><a href="#">경영공시</a></li>
        <li><a href="#">자체경영공시</a></li>
        <li><a href="#">참여알리미서비스</a></li>
        <li><a href="#"></a></li>
        <li><a href="#"></a></li>
        <li><a href="#"></a></li>
      </ul>
    </li>
    <li>
      <ul>
        <li><a href="#">해외진출 및 국제협력</a></li>
        <li><a href="#">정책연구</a></li>
        <li><a href="#">인터넷 진흥</a></li>
        <li><a href="#">인터넷 주소관리</a></li>
        <li><a href="#"></a></li>
        <li><a href="#"></a></li>
        <li><a href="#"></a></li>
      </ul>
    </li>
    <li>
      <ul>
        <li><a href="#">고객현장</a></li>
        <li><a href="#">청렴윤리</a></li>
        <li><a href="#">내부감사결과</a></li>
        <li><a href="#">고객만족경영</a></li>
        <li><a href="#">지속가능경영보고서</a></li>
        <li><a href="#">ARS서비스 안내</a></li>
        <li><a href="#">개인정보처리방침</a></li>
      </ul>
    </li>
    <li>
      <ul>
        <li><a href="#">공지사항</a></li>
        <li><a href="#">보도자료</a></li>
        <li><a href="#">사회공헌</a></li>
        <li><a href="#">포토뉴스</a></li>
        <li><a href="#"></a></li>
        <li><a href="#"></a></li>
        <li><a href="#"></a></li>
      </ul>
    </li>
  </ul>
</div>
```




3. UI/UX 관련 추가 지식

네비게이션

● 탭 메뉴/페이지 네비게이션

탭 메뉴 예제입니다.

EXAMPLE

인사채용 비위에 대한 취업제한 제도	조직도 및 관련예산	위원회 현황
법률고문 현황	노사 협의회 현황	

```
<div class="mbi_wrap">
<ul class="tab v2">
<li class="on" "><span>인사채용 비위에 대한 취업제한 제도</span></li>
<li><a href="#"><span>조직도 및 관련예산</span></a></li>
<li><a href="#"><span>위원회 현황</span></a></li>
<li class="tab_half bl"><a href="#"><span>법률고문 현황</span></a></li>
<li class="tab_half" ><a href="#"><span>노사 협의회 현황</span></a></li>
</ul>
</div>
```

페이징 예제입니다.

EXAMPLE



```
<div id="pageNavigation" class="pagewrap m4">
<div class="paging">
<span><a href="#" class="btn_home">처음 페이지로 이동</a> <a href="#" class="btn_prev">이전 페이지로 이동</a></span>
<span class="number"><strong><span>1</span></strong></span>
<a href="#" class="btn_paging"><span>2</span></a>
<a href="#" class="btn_paging"><span>3</span></a>
<a href="#" class="btn_paging"><span>4</span></a>
<a href="#" class="btn_paging"><span>5</span></a>
<a href="#" class="btn_paging"><span>6</span></a>
<a href="#" class="btn_paging"><span>7</span></a>
<a href="#" class="btn_paging"><span>8</span></a>
<a href="#" class="btn_paging"><span>9</span></a>
<a href="#" class="btn_paging"><span>10</span></a></span>
<span><a href="#" class="btn_next">다음 페이지로 이동</a>
<a href="#" class="btn_end">마지막 페이지로 이동</a></span>
</div>
</div>
```



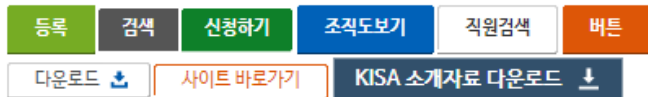
3. UI/UX 관련 추가 지식

버튼

● 버튼 예

컬러, 크기별 버튼 예제입니다.

EXAMPLE



```
<a href="#" class="btn1 green"> <span class="bg_green">등록</span></a>
<a href="#" class="btn1 gray"> <span class="bg_gray">검색</span></a>
<a href="#" class="btn1 green2"> <span class="bg_green2">신청하기</span></a>
<a href="#" class="btn1 blue"> <span class="bg_blue">조직도보기</span></a>
<a href="#" class="btn1 white"> <span class="bg_white">직원검색</span></a>
<a href="#" class="btn1 org"> <span class="bg_org">버튼</span></a>
<a href="#" class="btn1 down v2"> <span class="bg_down v2">다운로드</span></a>
<a href="#" class="btn1 white_m2 v2"> <span class="bg_white_m2">사이트 바로가기</span></a>
<a href="#" class="btn1 down v4"> <span class="bg_down v4">KISA 소개자료 다운로드</span></a>
```

3. UI/UX 관련 추가 지식

테이블

- 테이블 1은 `.tbl_lst`로 사용하며 테이블 2는 `.bbs_lst`로 사용합니다.
- th색상은 `green, blue, org`로 테이블 클래스 뒤에 붙여 사용합니다.

EXAMPLE

테이블 1

번호	제목	게시일	조회	첨부
3	가나다라 N	2015-05-28	398	
2	가나다라마바사 N	2015-05-28	5457	H
1	가나다라마바사아자차카타파하	2015-05-28	3519	H

테이블 2

번호	제목	게시일	조회	첨부
3	가나다라 N	2015-05-28	398	
2	가나다라마바사 N	2015-05-28	5457	H
1	가나다라마바사아자차카타파하	2015-05-28	3519	H

테이블 2 주황색

번호	제목	게시일	조회	첨부
3	가나다라 N	2015-05-28	398	
2	가나다라마바사 N	2015-05-28	5457	H
1	가나다라마바사아자차카타파하	2015-05-28	3519	H

3. UI/UX 관련 추가 지식

테이블

- 테이블 1은 .tbl_1st로 사용하며 테이블 2는 .bbs_1st로 사용합니다.
- th색상은 green, blue, org로 테이블 클래스 뒤에 붙여 사용합니다.

```
<table class="tbl_1st green">
<colgroup>
<col class="dn_mbi">
<col>
<col>
<col class="dn_mbi">
<col class="dn_mbi">
</colgroup>
<thead>
<tr>
<th scope="col" class="dn_mbi" style="width: 8%">번호</th>
<th scope="col" class="">제목</th>
<th scope="col" class="wt26">게시일</th>
<th scope="col" class="dn_mbi" style="width: 8%">조회</th>
<th scope="col" class="dn_mbi" style="width: 8%">첨부</th>
</tr>
</thead>
<tbody>
<tr>
<td class="dn_mbi">3</td>
<td class="lft"><a href="#"> 가나다라 </a><span class="ico ico_new"></span></td>
<td>2015-05-28</td> <td class="dn_mbi">398</td>
<td class="dn_mbi"></td> </tr> <tr> <td class="dn_mbi">2</td>
<td class="lft"><a href="#"> 가나다라마바사 </a><span class="ico ico_new"></span></td>
<td>2015-05-28</td> <td class="dn_mbi">5457</td>
<td class="dn_mbi"><span class="glyphicon glyphicon-floppy-disk"></span></td> </tr>
<tr>
<td class="dn_mbi">1</td>
<td class="lft"><a href="#"> 가나다라마바사아자차카타파하 </a></td>
<td>2015-05-28</td> <td class="dn_mbi">3519</td>
<td class="dn_mbi"><span class="glyphicon glyphicon-floppy-disk"></span></td> </tr>
</tbody>
</table>
```

3. UI/UX 관련 추가 지식

블릿

EXAMPLE

◦정보공개 청구권자

- 모든 국민
 - 미성년자, 재외국민, 수형인 등 포함
- 법인
 - 사법상의 사단법인/재단법인, 공법상의 법인(자치단체 포함), 정부투자기관, 정부출연기관 등
 - 법인격 없는 단체나 기관 포함
- 외국인
 - 국내에 일정한 주소를 두고 거주하는 자
 - 학술 연구를 위하여 일시적으로 체류하는 자
 - 국내에 사무소를 두고 있는 법인 또는 단체

```
<h3 class="bullet_large">정보공개 청구권자</h3>
<br>
<h4 class="bullet_medium">모든 국민</h4>
<br>
<h5 class="bullet_small">미성년자, 재외국민, 수형인 등 포함</h5>
<br>
<h4 class="bullet_medium">법인</h4>
<br>
<h5 class="bullet_small">사법상의 사단법인/재단법인, 공법상의 법인(자치단체 포함), 정부투자기관, 정부출연기관 등</h5>
<br>
<h5 class="bullet_small">법인격 없는 단체나 기관 포함</h5>
<br>
<h4 class="bullet_medium">외국인</h4>
<br>
<h5 class="bullet_small">국내에 일정한 주소를 두고 거주하는 자</h5>
<br>
<h5 class="bullet_small">학술 연구를 위하여 일시적으로 체류하는 자</h5>
<br>
<h5 class="bullet_small">국내에 사무소를 두고 있는 법인 또는 단체</h5>
```



3. UI/UX 관련 추가 지식

툴팁/폼

블릿 텍스트 예제입니다.

EXAMPLE



한국인터넷진흥원 정보공개 안내입니다.

공공기관이 작성 또는 취득하여 관리하고 있는 정보를 국민의 청구에 의하여 공개여부를 결정한 후 청구인에게 통지한 공개일시·장소에서 원본으로 공개합니다.

```
<div class="section v30 box_data">
  <span class="sp_ico2 ico_data"></span>
  <div class="txt_data v6">
    <p class="txt_space_v6 ft14">
      <strong><span class="clr_blue_v2">한국인터넷진흥원 정보공개 안내입니다.</span></strong>
    </p>
    <p class="txt_space_v7">공공기관이 작성 또는 취득하여 관리하고 있는 정보를 국민의 청구에 의하여 공개여부를 결정한 후 청구인에게 통지한 공개일시·장소에서 원본으로 공개합니다.
    </p>
  </div>
</div>
```



3. UI/UX 관련 추가 지식

툴팁/폼

검색 폼 예제입니다.

EXAMPLE

```
<div class="lst_field_v1">
<div class="srch_v1">
  <select id="listSearch" name="ST" value="" title="검색조건 선택" class="loc_selc" style="width: 105px">
    <option value="T">제목</option> <option value="C">내용</option>
    <option value="TC">제목+내용</option>
  </select>
  <span class="space_srch"><input type="text" name="SV" title="검색어 입력" value="" style="width: 100%"></span>
  <a href="javascript:fnSubmit();" class="btn1 gray loc_srch"><span class="bg_gray">검색</span></a>
</div>
</div>
```

작은 검색 폼 예제입니다.

EXAMPLE

```
<div class="group_search">
<form name="searchForm" method="get" action="#" target="_blank" >
  <fieldset>
    <span><input type="text" id="srch" name="keyword" placeholder="검색어를 입력해주세요." title="검색어 입력"></span>
    <a href="#" id="srchBtn" class="sp_ico_btn srch"></a>
  </fieldset>
</form>
</div>
```



3. UI/UX 관련 추가 지식

툴팁/폼

검색 폼 예제입니다.

EXAMPLE

```
<div class="lst_field_v1">
<div class="srch_v1">
<select id="listSearch" name="ST" value="" title="검색조건 선택" class="loc_selc" style="width: 105px">
<option value="T">제목</option> <option value="C">내용</option>
<option value="TC">제목+내용</option>
</select>
<span class="space_srch"><input type="text" name="SV" title="검색어 입력" value="" style="width: 100%"></span>
<a href="javascript:fnSubmit();" class="btn1 gray loc_srch"><span class="bg_gray">검색</span></a>
</div>
</div>
```

작은 검색 폼 예제입니다.

EXAMPLE

```
<div class="group_search">
<form name="searchForm" method="get" action="#" target="_blank" >
<fieldset>
<span><input type="text" id="srch" name="keyword" placeholder="검색어를 입력해주세요." title="검색어 입력"></span>
<a href="#" id="srchBtn" class="sp_ico_btn srch"></a>
</fieldset>
</form>
</div>
```




3. UI/UX 관련 추가 지식

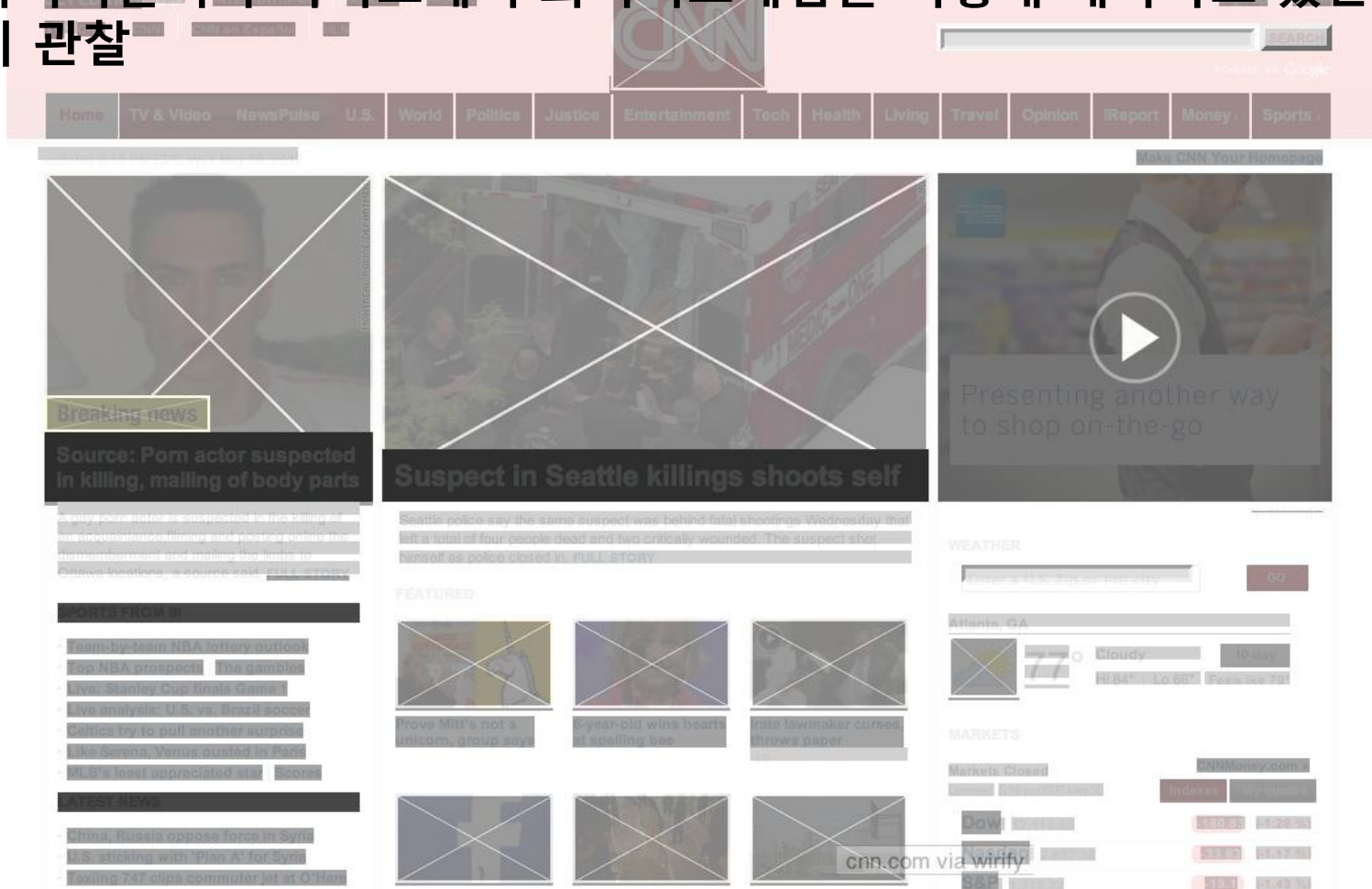
와이어프레임

- 의의
 - 정보 체계를 잡을 수 있음
 - 픽셀 레이어(포토샵 작업) 작업과 코드 작성 방향을 제시
 - 사용자가 인터페이스와 상호작용하는 방식을 정함
 - 버튼과 메뉴 동작을 담음
 - 레이아웃과 인터페이스에서의 상호작용을 설계

3. UI/UX 관련 추가 지식

와이어프레임

- 1단계: 영감 얻기
- 디자이너들이나 사이트에서 와이어프레임을 어떻게 제작하고 있는지 꾸준히 관찰





3. UI/UX 관련 추가 지식

와이어프레임

- 2단계: 과정 설계
- 디자인은 고유한 과정이라서 다양한 디자이너들은 와이어프레임을 제작하고 실제 디자인이나 코드로 전환하는 과정에 각각 달리 접근
- 전형적인 진행 과정

SKETCH > WIREFRAME > VISUAL > CODE

SKETCH > WIREFRAME > HI-DEF WIREFRAME > VISUAL > CODE

WIREFRAME > HI-DEF WIREFRAME > VISUAL > CODE

SKETCH > CODE

WIREFRAME > INTERACTIVE PROTOTYPE > VISUAL > CODE



3. UI/UX 관련 추가 지식

와이어프레임

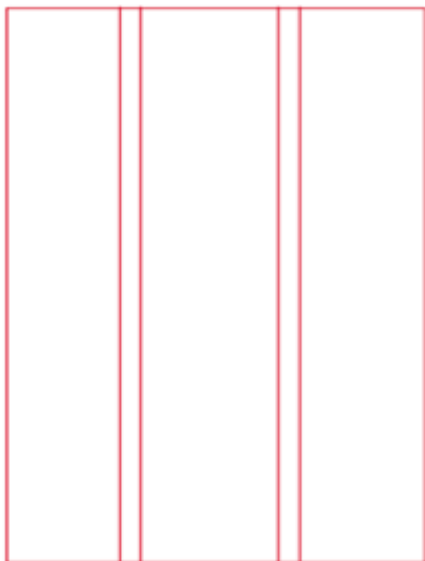
- 3단계: 툴 선택
 - Balsamiq
 - Omnigraffle
 - Axure
 - Flairbuilder
 - 온라인 애플리케이션
 - mockflow와 [hotgloo](#)와 mockingbird
 - 키노트/파워포인트
 - 어도비 CS
 - 파이어웍스
 - 일러스트레이터
 - 인디자인
 - 포토샵



3. UI/UX 관련 추가 지식

와이어프레임

- 4단계: 그리드 세팅
 - 디자인의 모든 요소, 즉 타이포그래피, 사진, 그림 등을 디자인 결과물로 융화시킬 수 있는 요소이며, 디자인에 질서를 도입하는 수단이다.
 - 웹 사이트가 시각적 질서와 일관성을 유지할 수 있어 보기에도 좋지만 사용하기에도 편해진다.
 - 칼럼 그리드
 - 페이지에 열을 여러 개 배치할 때 사용하는 그리드이다.

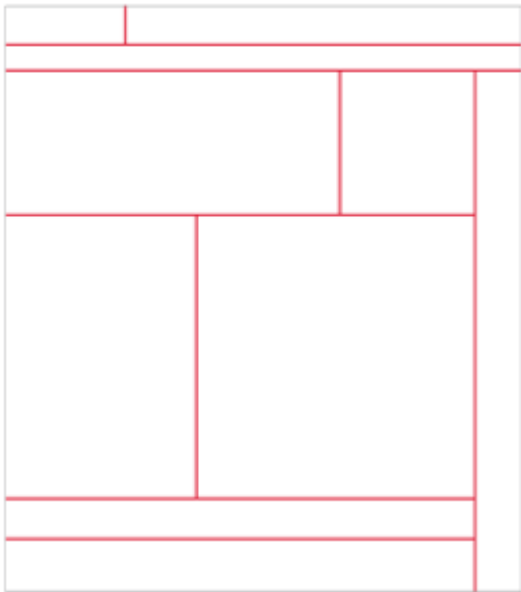




3. UI/UX 관련 추가 지식

와이어프레임

- 4단계: 그리드 세팅
- 계층 그리드
 - 웹 사이트에서 일반적으로 발견되는 그리드로 콘텐츠 정리형 그리드라고도 부른다.

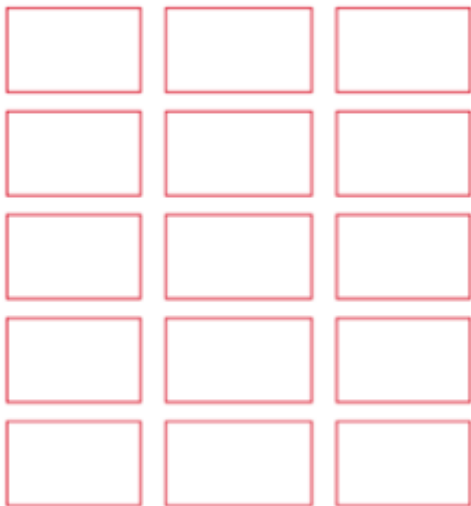




3. UI/UX 관련 추가 지식

와이어프레임

- 4단계: 그리드 세팅
 - 박스.리스트형 그리드
 - 바둑판 모양 그리드이다. 주로 포트폴리오나 사진 이미지를 보여 줄 때 사용된다.

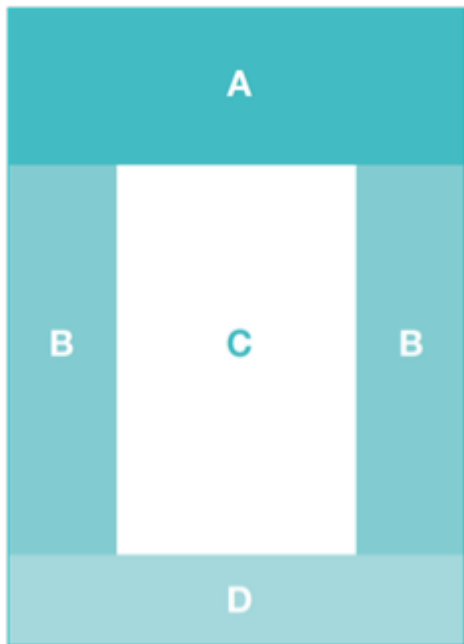




3. UI/UX 관련 추가 지식

와이어프레임

- 5단계: 박스(boxes)로 레이아웃 정하기
- 레이아웃의 영역별 구분
 - A는 헤더, B와 C는 콘텐츠 영역, D는 푸터 영역이다.



- 방문자에게 보여주고픈 정보의 순서를 왼쪽에서 오른쪽으로, 위에서 아래로 디자인의 기초원리 중 하나인 F와 Z패턴은 우리의 눈이 웹사이트를 읽는 흐름을 반영



3. UI/UX 관련 추가 지식

와이어프레임

● 5단계: 박스(boxes)로 레이아웃 정하기

• F 패턴

- 글을 왼쪽에서 오른쪽으로 읽는 대부분의 문화권에서 격자무늬를 사용해 가장 읽기 쉽고 편한 배열로 꼽히는 패턴
- 주로 제목 아래 간단한 설명을 붙인 사진을 넣고, 그 밑으로 콘텐츠를 소개하는 형식
- F 패턴은 다양하고 많은 정보를 전달하기보다는, 광고를 포함하거나 유저들의 참여를 필요로 하는 웹페이지에 매우 유용
- 또한 사이드바 영역을 잘 활용하면 유저들이 더 많이 콘텐츠에 상호작용하도록 만들 수도 있음

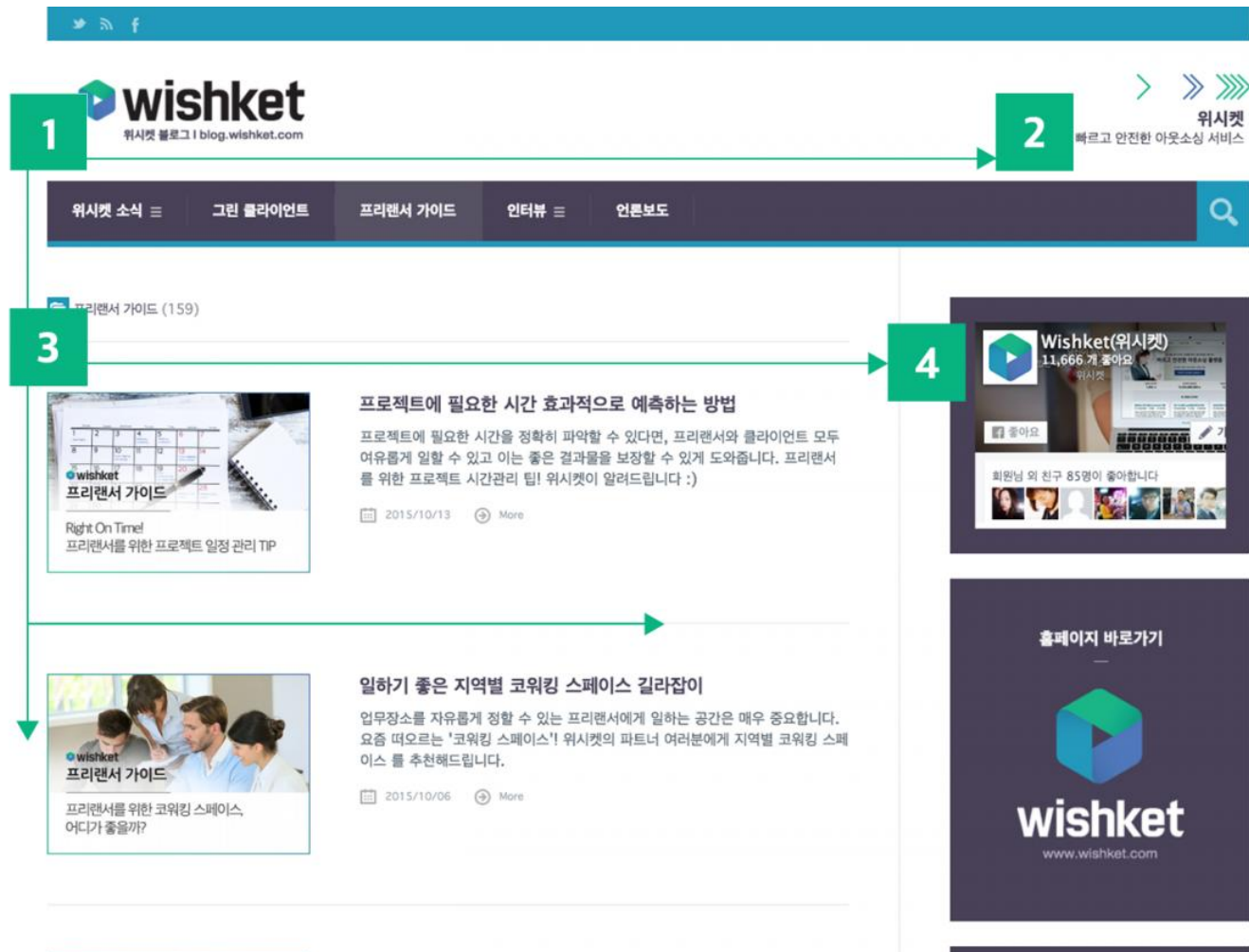
• Z 패턴

- Z패턴은 우리나라에선 흔히 '갈지(之)자 구도'라고 불리는 패턴
- 텍스트가 적고 독자가 어떤 반응을 할 지 명확히 예상할 때 적합
- 한 눈에 들어오기 때문에 유저가 페이지의 핵심에 빠르게 반응하도록 만들어준다는 장점
- 웹페이지에서 빠르게 반응을 이끌어내는 것은 생명

3. UI/UX 관련 추가 지식

와이어프레임

- 5단계: 박스(boxes)로 레이아웃 정하기
- F 패턴





3. UI/UX 관련 추가 지식

와이어프레임

- 5단계: 박스(boxes)로 레이아웃 정하기
- Z 패턴





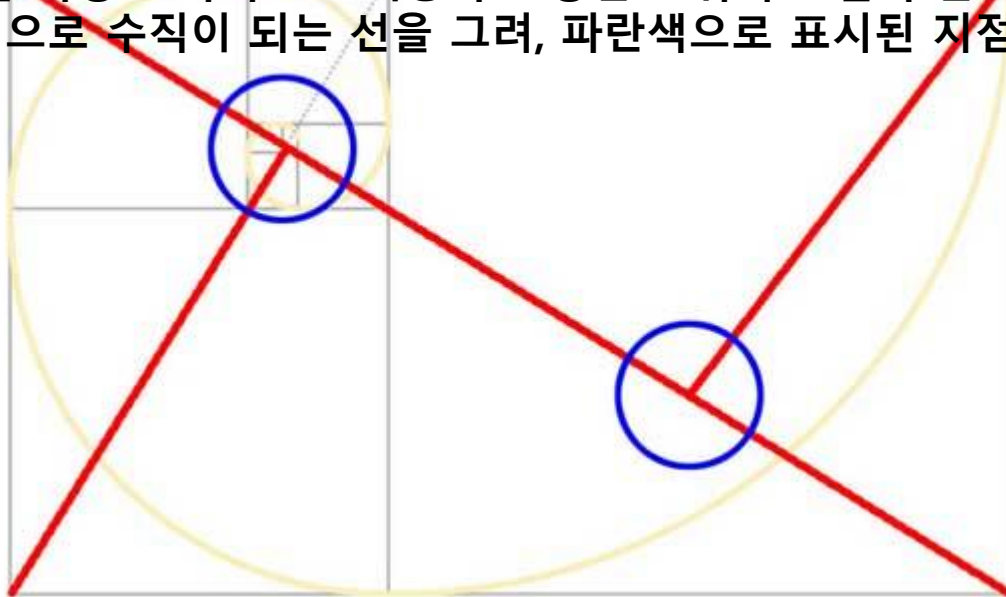
3. UI/UX 관련 추가 지식

와이어프레임

- 5단계: 박스(boxes)로 레이아웃 정하기

- 황금비율의 법칙

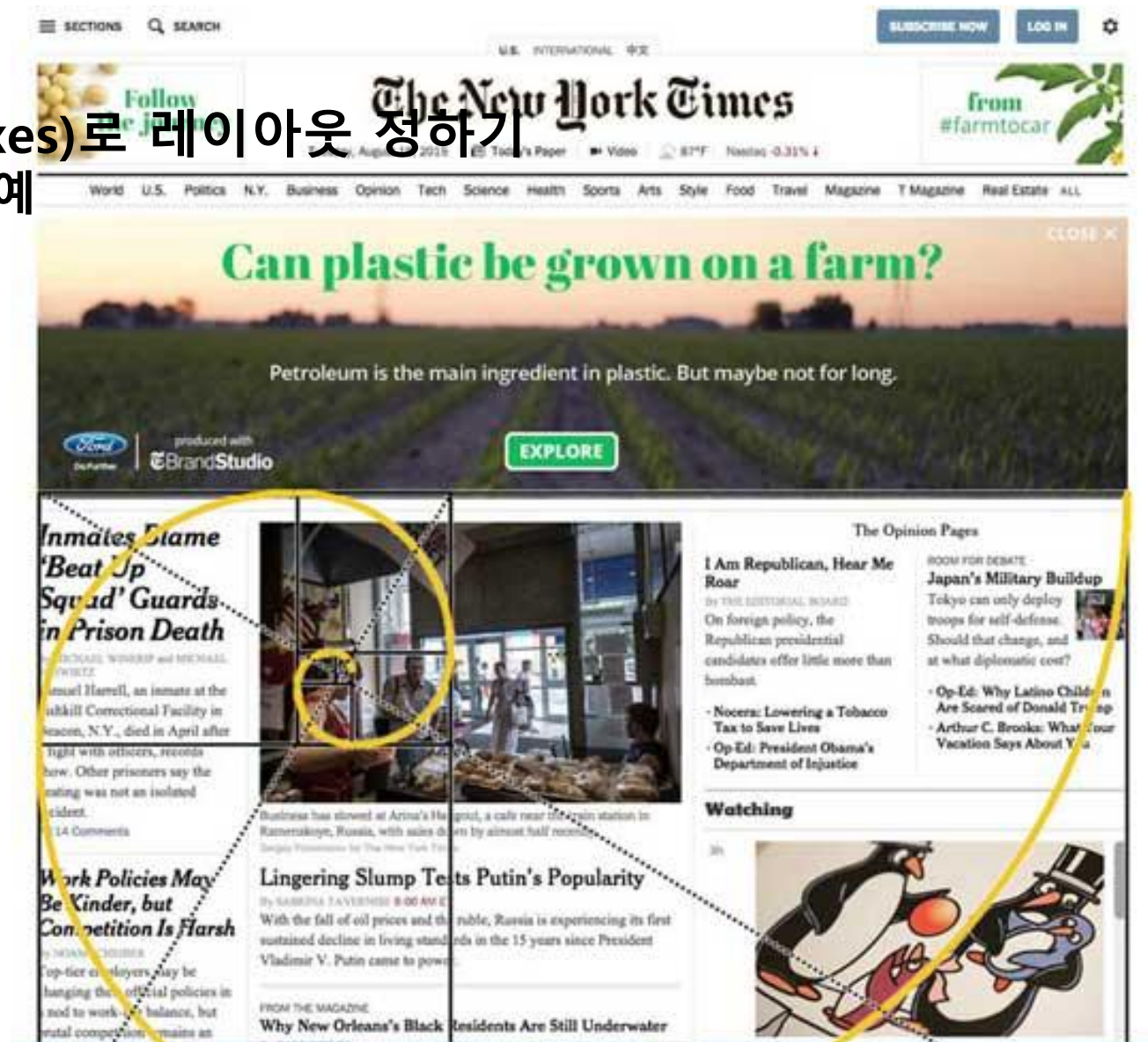
- Understand Golden Ratio
- 1:1.618이라는 비율이 핵심인 이 기술은, 가장 눈길을 끄는 부분을 찾는 데 도움을 주며, 시선을 원하는 위치로 유도하는 방법을 알려
- 웹페이지에 황금사각형을 겹쳐 놓고, 중요한 텍스트나 UI, 배너 등을 배치할 때 참고
- 방대한 콘텐츠를 다루는 뉴욕타임즈와 같은 뉴스 웹사이트는 이러한 황금 사각형의 법칙을 잘 활용
- 황금사각형을 가장 효과적으로 이용하는 방법은 위의 그림과 같이 대각선을 긋고 나머지 두 꼭지점으로 수직이 되는 선을 그려, 파란색으로 표시된 지점에 집중적으로 디자인



3. UI/UX 관련 추가 지식

와이어프레임

- 5단계: 박스(boxes)로 레이아웃 정하기
- 황금비율의 법칙 예



3. UI/UX 관련 추가 지식

와이어프레임

- 5단계: 박스(boxes)로 레이아웃 정하기
- 삼등분의 법칙(Understand the Rule of Thirds)
 - 수평과 수직방향으로 화면을 삼등분 하면 상-중-하, 좌-중-우 로 나뉘게 됨
 - 이 분할을 통해 중요한 부분이 좀 더 명확해지는 효과를 얻을 수 있음



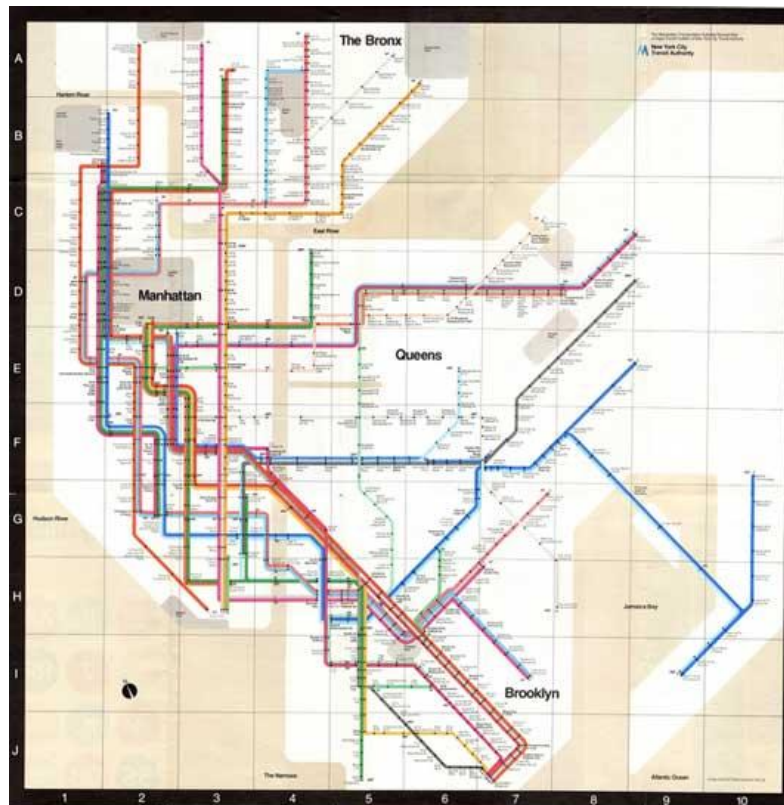
- A. 자연스레 가장 중앙에 시선이 가장 많이 향하는 동시에, 단순한 형태이기 때문에 주변 정보와 서로 충돌하지 않음
- B. 상단의 가격정보도 시선이 감
- C. 우측 상단의 일그러진 표정의 점원은 잘 눈에 띄지 않음
- D. 하단에 위치한 맛있어 보이는 음식과 식재료에 대한 정보, 그리고 기업의 사회적 공헌이 한눈에 들어옴
- E. 좌측 상단의 메뉴를 통해 세부적인 음식 종류들과 지점들의 위치 등의 정보 역시 쉽게 확인할 수 있음



3. UI/UX 관련 추가 지식

와이어프레임

- 5단계: 박스(boxes)로 레이아웃 정하기
 - 그리드
 - 그리드(Grid)란, 콘텐츠를 구성할 때 가이드 라인으로 사용되는 다양한 분할선
 - 1972년에 마시모 비넬리(Massimo Vignelli)가 디자인한 뉴욕시의 지하철 노선도





3. UI/UX 관련 추가 지식

와이어프레임

- 6단계: 정보 체계 정의
- 박스 배열 방식이 흡족하다면, 정보의 체계화가 잘 되었는지 감 잡도록 콘텐츠의 아주 작은 부분을 넣는다

LOGO	HOME	FEATURES	ABOUT US	SUPPORT	BUY NOW
Introducing the world's best application					
DOWNLOAD FREE TRIAL Try out without obligations for 30 days!					
Save time	So simple to use	Cloud integration			
Read our blog	Send us feedback	Get to know us			



3. UI/UX 관련 추가 지식

와이어프레임

- 6단계: 정보 체계 정의
 - UX Choreography (UX 연출)
 - ① 공간 인식 spatial awareness과 배치 원리'(principle of staging)
 - ② 사용자 포커스 User Focus와 목표의 '명확함(Clarity)'
 - ③ 브랜드의 톤 Brand Tone과 요소들의 조화

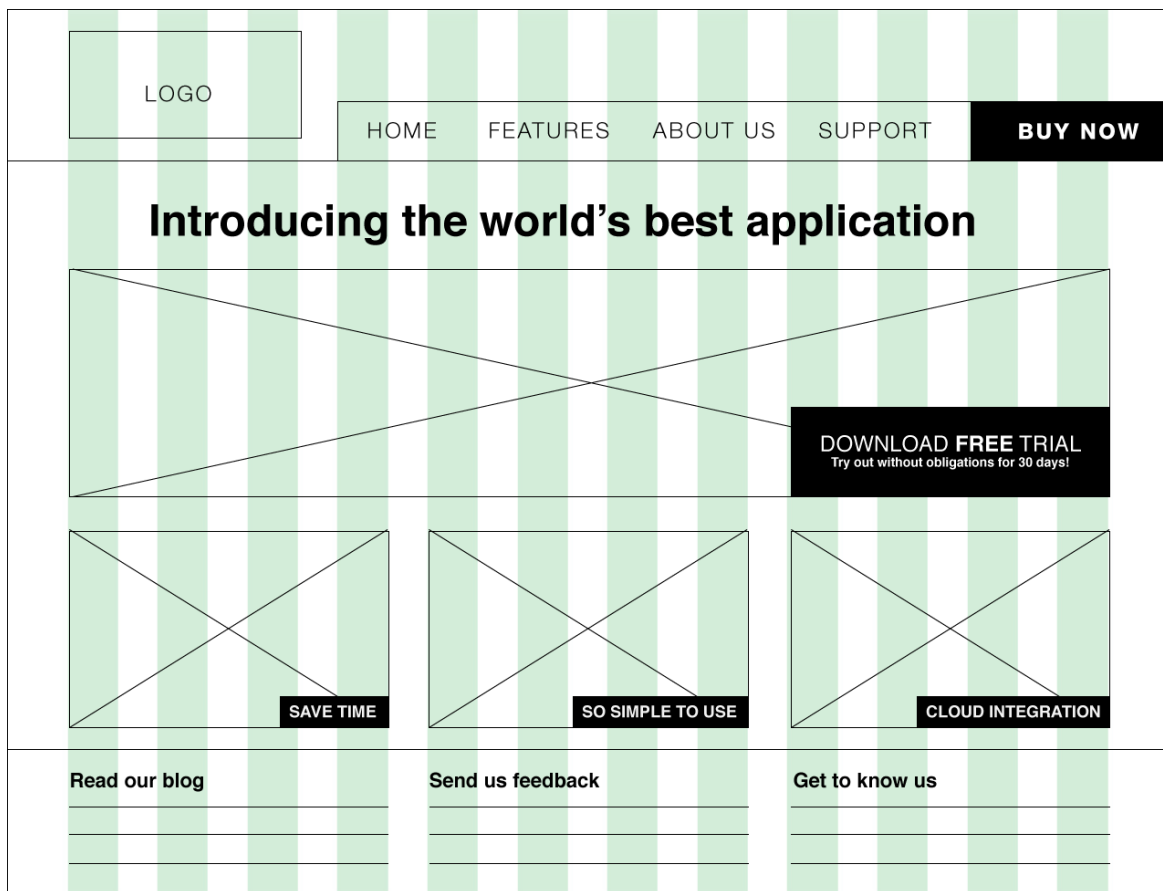
모든 것은 하나의 테마를 따르게 된다.



3. UI/UX 관련 추가 지식

와이어프레임

- 6단계: 정보 체계 정의
 - 스크린샷으로 효과를 높이고, 웹사이트에서 시각적으로 중요한 영역을 지정하는데 검은색 박스를 사용 예

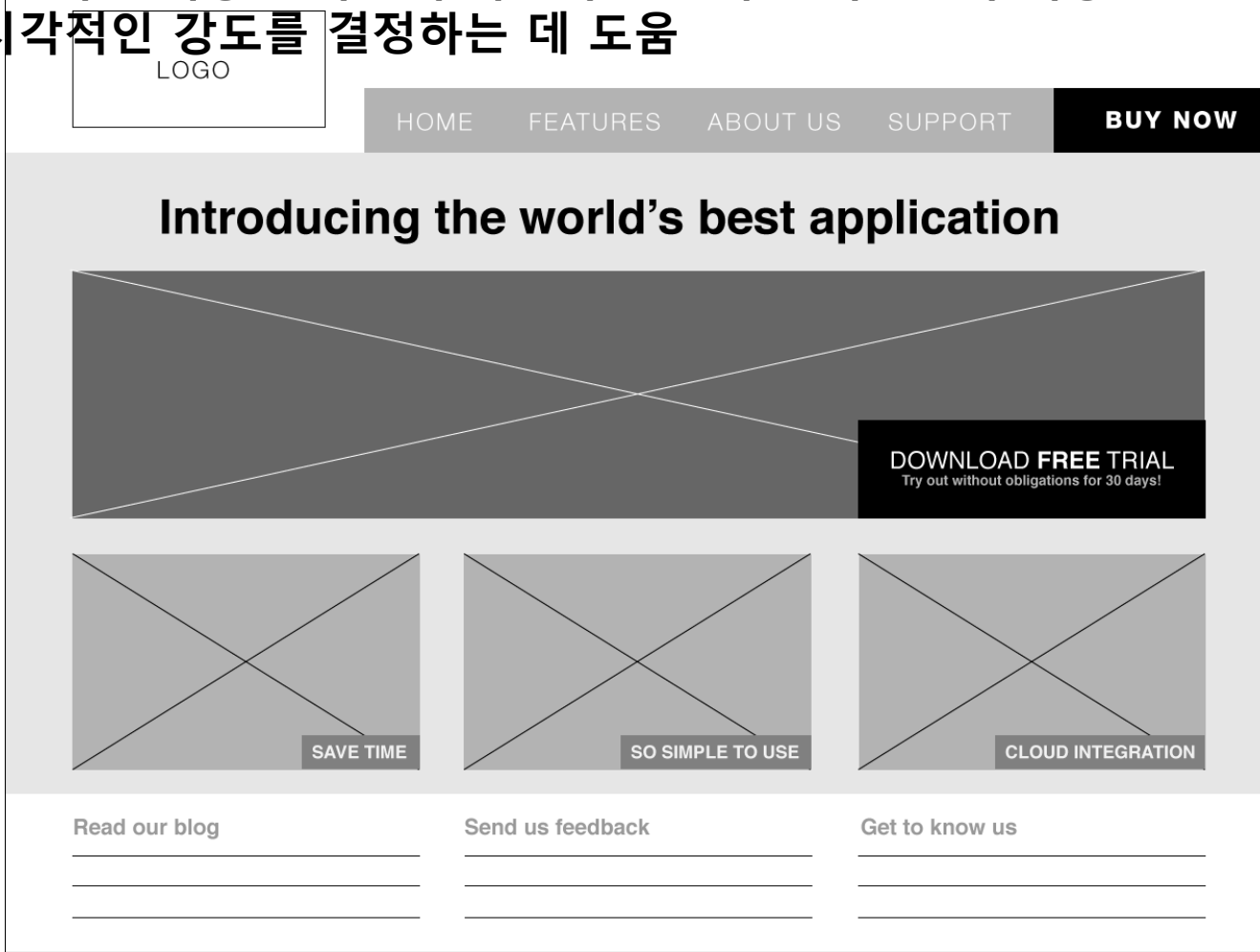




3. UI/UX 관련 추가 지식

와이어프레임

- 7단계: 그레이스케일을 이용한 세부 조정
- 그레이스케일 색상을 폭넓게 사용하면 컬러 팔레트에서 색상을 고르지 않아도 요소의 시각적인 강도를 결정하는 데 도움





3. UI/UX 관련 추가 지식

와이어프레임

- 8단계: 고해상도 와이어프레임
- 선택 가능한 단계

Home
frontpage

About
who we are

Services
what we offer

Case Studies
what we've done

Testimonials
kudos by our clients

F.A.Q
questions?

Say Hello
drop us a note or a RFQ

webwise
SOLUTIONS

featured case study

jquery slide panel (3 rotating case studies?)

building great communities powered by drupal

Introducing...

WebWise Solutions is dedicated to one goal: making your customers insanely happy with your products.

Everything we do is designed to attract, compel, delight, tantalize, and educate people so they can't imagine life without you.

How do we do it? We have spent the last twelve years creating corporate web sites and web communities, and have learned what works and what doesn't work. [more](#)

Clients we serve



What we specialise in

We build engaging communities powered by the powerful open-source software, Drupal.

- [Full Services Packages](#)
- [Editorial and Community Management Only](#)
- [Communities On-Demand](#)

selected testimonial

Put us to work for
you. We live to serve.

Use our Talk to Us form to drop
us a note.

Advertising opportunities

Helping clients with their communities gives us the occasional opportunity to match up advertisers and available space. [Click here to view the inventory we are currently helping to fill.](#)

FAQ

Q: What's the difference between a web community and an online forum?

A: This is the most common question we run into with our clients. As we consult with them about their needs, they'll often say, "Oh, we already have a forum, so I guess that's our community."....[see full answer](#)

Home

About

Services

Case Studies

© WebWise Solutions 2009

Terms, Conditions, and Notices.

Testimonials

Say Hello

FAQ

3. UI/UX 관련 추가 지식

와이어프레임

- 9단계: 와이어프레임을 실제 화면 디자인으로 전환
- 와이어프레임을 시각적인 디자인으로 바꾼 예제

[Home](#) [About](#) [Services](#) [Case Studies](#) [Testimonials](#) [F.A.Q](#) [Say Hello](#)
frontpage who we are what we offer what we've done kudos by our clients questions? drop us a note or a RFQ



building great communities powered by drupal

featured case study

jquery slide panel (3 rotating case studies?)

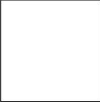
Introducing...

WebWise Solutions is dedicated to one goal: making your customers insanely happy with your products.

Everything we do is designed to attract, compel, delight, tantalize, and educate people so they can't imagine life without you.

How do we do it? We have spent the last twelve years creating corporate web sites and web communities, and have learned what works and what doesn't work. [more](#)

Clients we serve



What we specialise in

We build engaging communities powered by the powerful open-source software, Drupal.

- Full Services Packages
- Editorial and Community Management Only
- Communities On-Demand

selected testimonial

Put us to work for you. We live to serve.

Use our Talk to Us form to drop us a note.

Advertising opportunities

Helping clients with their communities gives us the occasional opportunity to match up advertisers and available space. [Click here to view the inventory we are currently helping to fill.](#)

FAQ

Q: What's the difference between a web community and an online forum?

A: This is the most common question we run into with our clients. As we consult with them about their needs, they'll often say, "Oh, we already have a forum, so I guess that's our community."...[see full answer](#)

Home

About

Services

Case Studies

© WebWise Solutions 2009

Terms, Conditions, and Notices.

Testimonials

Say Hello

FAQ

[Home](#) [About](#) [Services](#) [Case Studies](#) [Testimonials](#) [F.A.Q](#) [Say Hello](#)
frontpage information what we offer what we've done kudos by clients questions? drop us a note or a RFQ



building great communities POWERED BY DRUPAL

featured case studies

hpsymantec.com

Sun and Symantec are part of a strategic relationship formed to help the two global leaders sell complementary products from each other's portfolios.



Introducing

WebWise Solutions is dedicated to one goal: making your customers insanely happy with your products.

Everything we do is designed to attract, compel, delight, tantalize, and educate people so they can't imagine life without you.

How do we do it? We have spent the last twelve years creating corporate web sites and web communities, and have learned what works and what doesn't work. [more](#)

Clients we serve



What we specialize in

We build engaging communities powered by the powerful open-source software, Drupal.

- Full Services Packages
- Editorial and Community Management Only
- Communities On-Demand

Scott Jones, Altiris product manager

“ In the IT industry, building an online community around a product is key to its long-term success. The people at WebWise are masters at using the Internet to deliver product success.

Put us to work for you. We live to serve.

[drop us a note.](#)

Advertising opportunities

Helping clients with their communities gives us the occasional opportunity to match up advertisers and available space. [Click here to view the inventory we are currently helping to fill.](#)

F.A.Q

Q: What's the difference between a web community and an online forum?

A: This is the most common question we run into with our clients. As we consult with them about their needs, they'll often say, "Oh, we already have a forum, so I guess that's our community."...[see full answer](#)

Home

About

Services

Case Studies

© WebWise Solutions 2009

Terms, Conditions, and Notices.

Testimonials

Say Hello

FAQ

4. 실습

실습 1: HTML 예제 만들어 보기

클라이언트 (Client)

- 사용자가 직접 보고 조작하는 부분
- 웹 브라우저, 모바일 앱
- 역할: 사용자 입력 → 서버에 요청
- 구성 요소
 - HTML: 구조 (뼈대)
 - CSS: 디자인 (외관)
 - JavaScript: 동작 (기능)

『3과목』 최소구현 + 리뷰

- Part 1. 오리엔테이션
- Part 2. GREEN, Review Cursor AI 프롬프트
- Part 3. SW Implementation
- Part 4. GREEN 단계의 최소 구현
- Part 5. 종합 실습 & 마무리





1. GREEN 단계란?

| GREEN 단계란?

TDD의 3단계

1. RED – 실패하는 테스트를 먼저 작성
2. GREEN – 그 테스트만 통과시키는 최소한의 코드 작성
3. REFACTOR – 설계/중복/가독성을 개선

“GREEN 단계에서 minimally 구현할 때, 전통적인 구현과 뭐가 다른가?”



1. GREEN 단계란?

| GREEN 단계란?

GREEN 단계의 최소 구현(minimal implementation)

“테스트를 지금 당장 통과시키는 데 필요한 것만” 구현하는 것이고,
전통적인 구현은 “나중에 필요할 것까지 미리 설계·구현”하는 경향

2. “무엇을 기준으로 코드를 쓰는가?”

| 전통적인 구현

- 기준: 머릿속의 설계, 요구사항 문서, 경험, 감(추측)
- 흐름:
 - 요구사항 보고
 - 전체 구조와 API를 머릿속/종이에 대략 설계
 - “나중에 필요할 것 같은 기능”도 함께 넣으려고 함
- 결과:
 - 테스트는 나중에(혹은 수동 테스트)
 - “이 정도는 필요하겠지”라는 가정이 코드에 많이 섞임

2. “무엇을 기준으로 코드를 쓰는가?”

| GREEN 단계의 최소 구현

- 기준: 이미 RED 단계에서 작성해 둔 실패하는 테스트
- 흐름:
 - 이미 존재하는 테스트 하나(혹은 작은 그룹)를 바라보며
 - “이 테스트를 통과시키는 데 필요한 것만” 구현
- 결과:
 - 테스트가 “지금 당장 필요한 동작의 경계”를 정해줌
 - 테스트가 요구사항의 최소 단위 스펙 역할
- 차이:
 - 전통 구현은 “미래까지 포함한 설계”를 기준으로,
 - GREEN은 “지금 눈앞의 테스트”를 기준으로 움직입니다.



3. 개발 순서의 근본적 차이

전통적인 구현

1. 요구사항 분석



2. 설계 (완벽한 설계 추구)



3. 완전한 구현 (한 번에 모든 기능)



4. 테스트 작성



5. 버그 수정



3. 개발 순서의 근본적 차이

GREEN 단계의 최소 구현

1. 요구사항 분석



2. 실패하는 테스트 작성 (RED)



3. 최소 구현 (GREEN) ← 여기!



4. 리팩토링 (REFACTOR)



5. 다음 테스트로 반복



3. 코드 작성 범위(스코프)

전통적인 구현

- 한 번에
 - 기능 A 전체
 - 그 안의 서브 기능들
 - 확장 포인트까지
- 예:
 - 로그인 기능을 만들면서
 - 소셜 로그인 확장 포인트
 - 비밀번호 재설정
 - 계정 잠금 정책
 - 로그 이력 기록



3. 코드 작성 범위(스코프)

전통적인 구현

전통적 방식: 처음부터 모든 기능을 구현

```
class Calculator:
```

```
    """완벽한 계산기를 한 번에 구현"""
```

```
    def add(self, a: float, b: float) -> float:
```

```
        """덧셈"""
```

```
        if not isinstance(a, (int, float)):
```

```
            raise TypeError("a must be a number")
```

```
        if not isinstance(b, (int, float)):
```

```
            raise TypeError("b must be a number")
```

```
        return a + b
```

```
    def subtract(self, a: float, b: float) -> float:
```

```
        """뺄셈"""
```

```
        if not isinstance(a, (int, float)):
```

```
            raise TypeError("a must be a number")
```

```
        if not isinstance(b, (int, float)):
```

```
            raise TypeError("b must be a number")
```

```
        return a - b
```

```
    def multiply(self, a: float, b: float) -> float:
```

```
        """곱셈"""
```

```
        if not isinstance(a, (int, float)):
```

```
            raise TypeError("a must be a number")
```

```
        if not isinstance(b, (int, float)):
```

```
            raise TypeError("b must be a number")
```

```
        return a * b
```



3. 코드 작성 범위(스코프)

전통적인 구현

```
def divide(self, a: float, b: float) -> float:
    """나눗셈"""
    if not isinstance(a, (int, float)):
        raise TypeError("a must be a number")
    if not isinstance(b, (int, float)):
        raise TypeError("b must be a number")
    if b == 0:
        raise ZeroDivisionError("Cannot divide by zero")
    return a / b
```

```
def power(self, a: float, b: float) -> float:
    """거듭제곱"""
    if not isinstance(a, (int, float)):
        raise TypeError("a must be a number")
    if not isinstance(b, (int, float)):
        raise TypeError("b must be a number")
    return a ** b
```

```
def sqrt(self, a: float) -> float:
    """제곱근"""
    if not isinstance(a, (int, float)):
        raise TypeError("a must be a number")
    if a < 0:
        raise ValueError("Cannot calculate square root of negative number")
    return a ** 0.5
```




3. 코드 작성 범위(스코프)

GREEN 단계의 최소 구현

- 한 번에
 - 테스트가 요구하는 범위만 구현
- 예:
 - 로그인 기능에 대해 현재 테스트가 요구하는 건
 - “아이디/비밀번호가 정확하면 True를 반환한다” 뿐이라면
 - 이 경우,
 - 계정 잠금
 - 소셜 로그인
 - 로그 기록 전부 나중으로 미룸
- 차이
 - GREEN은 ****“Do the simplest thing that could possibly work”****를 철저히 지키기 때문에
 - 테스트에서 요구하지 않은 기능/확장은 의도적으로 구현하지 않습니다.



3. 코드 작성 범위(스코프)

GREEN 단계의 최소 구현

TDD GREEN 단계: "최소" 구현



RED: 첫 번째 실패 테스트

```
import pytest
```

```
class TestCalculator:
```

```
    def test_add_two_numbers(self):
```

```
        """두 숫자를 더하는 기능 테스트"""
```

```
        calc = Calculator()
```

```
        result = calc.add(2, 3)
```

```
        assert result == 5 # 실패! Calculator 클래스가 없음
```



GREEN: 테스트를 통과시키는 최소 코드

```
pythonclass Calculator:
```

```
    def add(self, a, b):
```

```
        return 5 # 가장 단순한 구현!
```



3. 코드 작성 범위(스코프)

GREEN 단계의 최소 구현

RED: 두 번째 테스트 추가

```
class TestCalculator:
    def test_add_two_numbers(self):
        calc = Calculator()
        assert calc.add(2, 3) == 5

    def test_add_different_numbers(self):
        calc = Calculator()
        assert calc.add(5, 7) == 12 # 실패! 항상 5 반환
```

GREEN: 실제 로직 추가

```
class Calculator:
    def add(self, a, b):
        return a + b # 이제 실제 덧셈 구현
```



3. 코드 작성 범위(스코프)

GREEN 단계의 최소 구현

RED: 예외 케이스 테스트

```
def test_add_with_string_raises_error(self):  
    calc = Calculator()  
    with pytest.raises(TypeError):  
        calc.add("2", 3) # 실패! TypeError가 발생하지 않음
```

GREEN: 예외 처리 추가

```
class Calculator:  
    def add(self, a, b):  
        if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):  
            raise TypeError("Both arguments must be numbers")  
        return a + b
```



4. 의사결정 시점의 차이

전통적인 구현

사전 설계 중심

모든 결정을 미리 해야 함

- ✓ 어떤 예외가 발생할 수 있는가?
- ✓ 어떤 검증이 필요한가?
- ✓ 어떤 성능 최적화가 필요한가?
- ✓ 확장성을 위해 무엇을 준비해야 하는가?
- ✓ 어떤 디자인 패턴을 사용할 것인가?

↓
한 번에 구현

↓
나중에 테스트

문제점:

- Over-engineering (과도한 설계)
- YAGNI 원칙 위반 (You Aren't Gonna Need It)
- 사용되지 않는 코드 작성
- 변경 비용 증가



4. 의사결정 시점의 차이

GREEN 단계의 최소 구현 점진적 의사결정

현재 필요한 것만 결정	
Q: 지금 이 테스트를 통과하려면 최소한 무엇이 필요한가?	

↓
최소 구현

↓
다음 테스트

↓
점진적 확장

4. 의사결정 시점의 차이

GREEN 단계의 최소 구현 점진적 의사결정

현재 필요한 것만 결정	
Q: 지금 이 테스트를 통과하려면 최소한 무엇이 필요한가?	

↓
최소 구현
↓
다음 테스트
↓
점진적 확장



4. 의사결정 시점의 차이

GREEN 단계의 최소 구현

1단계: 빈 문자열 처리

```
@pytest.fixture
def calculator():
    return StringCalculator()

def test_empty_string_returns_zero(calculator):
    assert calculator.add("") == 0

# GREEN: 최소 구현
class StringCalculator:
    def add(self, numbers: str) -> int:
        return 0 # 일단 0 반환
```




4. 의사결정 시점의 차이

GREEN 단계의 최소 구현

2단계: 단일 숫자 처리

```
def test_single_number_returns_value(calculator):  
    assert calculator.add("5") == 5
```

GREEN: 기능 확장

```
class StringCalculator:  
    def add(self, numbers: str) -> int:  
        if numbers == "":  
            return 0  
        return int(numbers) # 단일 숫자 처리
```



4. 의사결정 시점의 차이

GREEN 단계의 최소 구현

3단계: 두 숫자 처리

```
def test_two_numbers_returns_sum(calculator):  
    assert calculator.add("1,2") == 3
```

GREEN: 심표 구분 처리

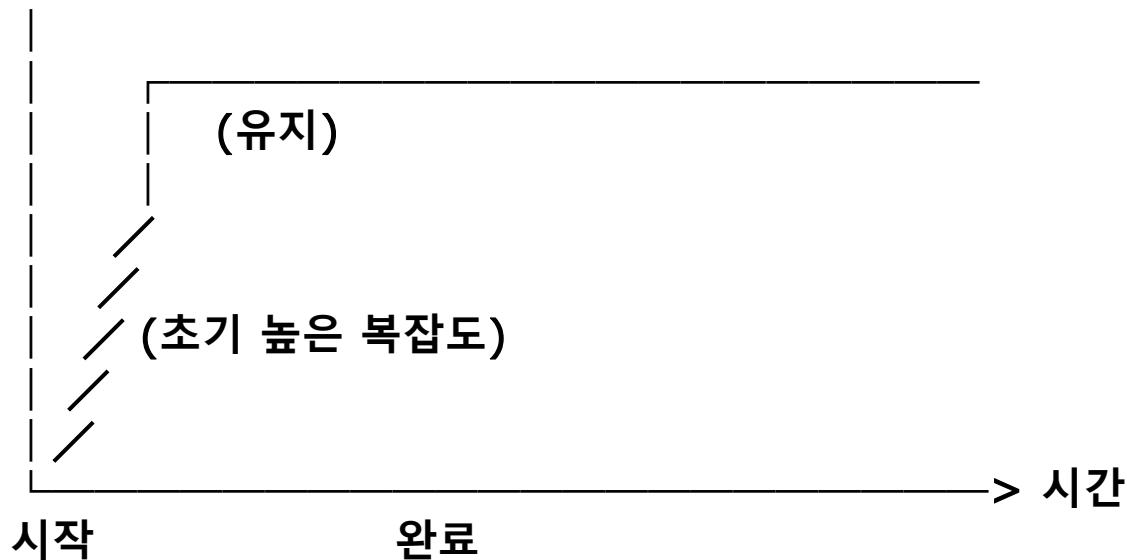
```
class StringCalculator:  
    def add(self, numbers: str) -> int:  
        if numbers == "":  
            return 0  
        if "," not in numbers:  
            return int(numbers)  
        parts = numbers.split(",")  
        return sum(int(x) for x in parts)
```



5. 코드 복잡도 증가 패턴

전통적인 구현

초기 복잡도 높음
복잡도



특징:

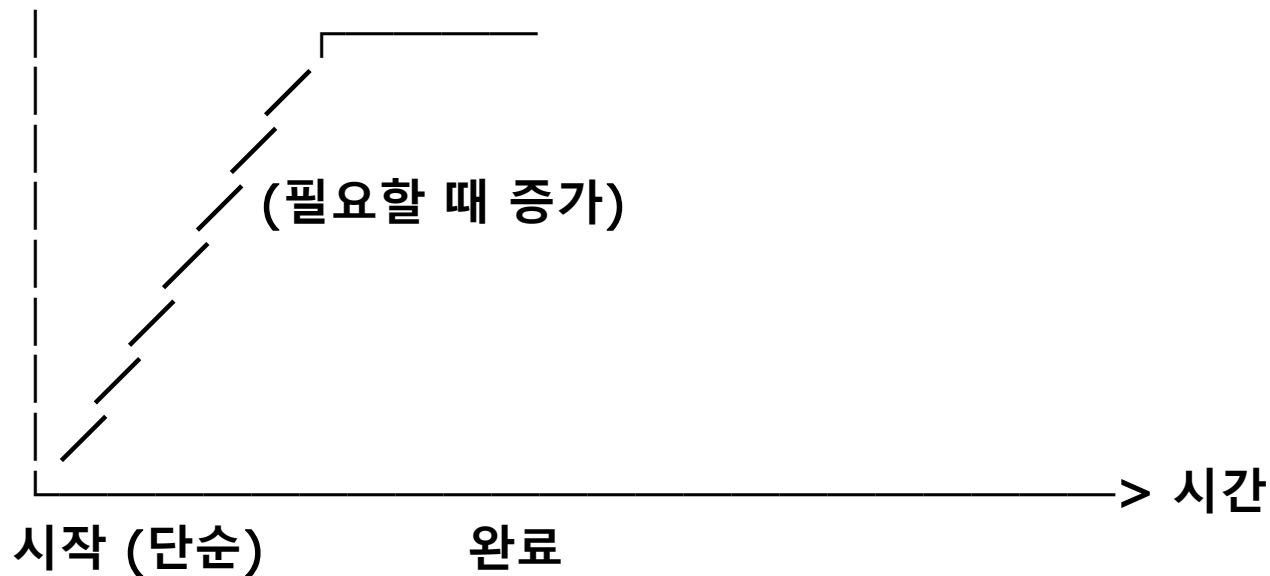
- 초기부터 복잡한 구조
- 이해하기 어려움
- 불필요한 추상화
- 테스트 작성 어려움



5. 코드 복잡도 증가 패턴

GREEN 단계의 최소 구현

점진적 복잡도 증가
복잡도



특징:

- 항상 단순한 상태 유지
- 필요할 때만 복잡도 추가
- 테스트로 안전성 보장
- 이해하기 쉬움



6. 설계/아키텍처에 대한 태도

전통적인 구현

- 처음부터 “예쁘게” 만들고 싶어 함:
 - 계층 구조 (Controller / Service / Repository / DTO / Domain)
 - 인터페이스 분리
 - DI 컨테이너, 패턴 적용 (Strategy, Factory, Observer...)
- 문제:
 - 아직 테스트된 사용 사례가 없는데 아키텍처부터 복잡해짐
 - 나중에 실제 요구사항과 어긋나서 다시 뜯어고칠 수 있음



6. 설계/아키텍처에 대한 태도

GREEN 단계의 최소 구현

- 목표:
 - 일단 돌아가게 만든 뒤, 설계는 REFACTOR 단계에서 정교화
- 원칙:
 - 우선은 직선적·직관적인 코드도 허용
 - 다만 “나중에 리팩토링하기 어려운 구조(하드코딩된 전역 상태 등)”는 피함
- 예:
 - 처음엔

```
def calculate_shipping(country, weight):  
    if country == "KR":  
        return weight * 1000  
    else:  
        return weight * 2000
```

- 테스트가 여러 국가/규칙을 추가할 때 REFACTOR에서
- 전략 패턴, 규칙 테이블, 정책 객체 등으로 분리
- 차이:
 - 전통 구현은 처음부터 아키텍처를 멋지게 짜려 하고,
 - GREEN은 먼저 “동작”을 확보한 뒤, 아키텍처는 나중에 REFACTOR에서 책임집니다.

7. 일반화 vs 구체화

| 전통적인 구현

- 습관:

- “이거 나중에 다른 데서도 쓸 것 같아”
- “재사용 가능하게 만들어야지”

- 그래서:

- 아직 쓰이지 않는 추상화, 인터페이스, 제네릭, 상속 구조가 많이 등장
- “YAGNI(You Aren’t Gonna Need It)”를 깨기 쉬움

7. 일반화 vs 구체화

GREEN 단계의 최소 구현

- 원칙:

- 테스트가 요구하는 수준으로만 일반화

- 예:

- 테스트가 지금은 Operation.ADD만 쓰고 있다면
- 최소 구현은 정말 극단적으로 이럴 수도 있음:

```
def calculate(x, y, operation):  
    return x + y # ADD에 대해서만 맞으면 GREEN OK
```

- 그 다음, SUBTRACT에 대한 테스트가 추가되면

```
if operation == Operation.ADD:  
    return x + y  
elif operation == Operation.SUBTRACT:  
    return x - y
```

- 계속 테스트가 늘어나면서
- → dict 매핑 / 전략 패턴 / Enum 기반 구조로 발전 (REFACTOR)

- 차이:

- GREEN은 **“필요할 때만 일반화”**하고,
- 전통 구현은 **“나중에 필요할 것 같아서 미리 일반화”**하는 경향.

8. 예외/에러 처리의 범위

| 전통적인 구현

- 한 번에:
 - 모든 경계/예외를 머리 속에서 상상하면서 처리하려고 함
 - “이 경우도 에러고, 저 경우도 에러고...”
- 결과:
 - 코드가 초반부터 매우 복잡해짐
 - 실제로는 안 쓰이는 예외 처리도 발생



8. 예외/에러 처리의 범위

전통적인 구현

- 전통적 방식: 선제적 예외 처리

```
def process_order(order_data: dict) -> Order:
```

```
    """주문 처리 - 모든 예외를 미리 처리"""
```

```
    # 수많은 사전 검증
```

```
    if order_data is None:
```

```
        raise ValueError("Order data cannot be None")
```

```
    if not isinstance(order_data, dict):
```

```
        raise TypeError("Order data must be a dictionary")
```

```
    if 'user_id' not in order_data:
```

```
        raise KeyError("user_id is required")
```

```
    if not isinstance(order_data['user_id'], int):
```

```
        raise TypeError("user_id must be an integer")
```

```
    if order_data['user_id'] <= 0:
```

```
        raise ValueError("user_id must be positive")
```

```
    if 'items' not in order_data:
```

```
        raise KeyError("items is required")
```

```
    if not isinstance(order_data['items'], list):
```

```
        raise TypeError("items must be a list")
```

```
    if len(order_data['items']) == 0:
```

```
        raise ValueError("items cannot be empty")
```

```
    # 실제 로직은 몇 줄 안됨
```

```
    order = Order(**order_data)
```

```
    order.save()
```

```
    return order
```

- 문제점:

- 코드의 90%가 검증 로직
- 실제로 발생하지 않을 수도 있는 예외 처리
- 가독성 저하
- 테스트로 검증되지 않음

8. 예외/에러 처리의 범위

GREEN 단계의 최소 구현

- 원칙:
 - 테스트가 정의한 예외 상황만 우선 처리
 - 새로운 예외 상황이 등장하면
 - ① RED: 그 상황에 대한 테스트를 먼저 작성
 - ② GREEN: 그 테스트를 통과하는 최소 예외 처리 추가
- 예:
 - 처음엔 0으로 나누는 경우를 전혀 처리하지 않다가
 - 테스트에

```
with pytest.raises(ZeroDivisionError):  
    calculator.calculate(10, 0, Operation.DIVIDE)
```

- 를 추가한 순간,

```
def _divide(self, x, y):  
    if y == 0:  
        raise ZeroDivisionError(...)  
    return x / y
```

- 를 GREEN에서 구현

- 차이:
 - 전통 구현은 머리 속에서 상상한 모든 에러를 한 번에 처리하려 하고,
 - GREEN은 테스트에 잡힌 에러만 점진적으로 처리합니다.

8. 예외/에러 처리의 범위

GREEN 단계의 최소 구현

● 실제 필요할 때만 추가

1단계: 정상 케이스만 구현

```
def test_create_order_with_valid_data():
    order_data = {'user_id': 1, 'items': [{'id': 1, 'qty': 2}]}
    order = process_order(order_data)
    assert order.user_id == 1
```

```
def process_order(order_data: dict) -> Order:
    order = Order(**order_data)
    order.save()
    return order
```

2단계: 실제 발생한 버그 수정

```
def test_create_order_with_empty_items_raises_error():
    order_data = {'user_id': 1, 'items': []}
    with pytest.raises(ValueError, match="items cannot be empty"):
        process_order(order_data)
```

```
def process_order(order_data: dict) -> Order:
    if not order_data.get('items'): # 필요할 때 추가
        raise ValueError("items cannot be empty")
    order = Order(**order_data)
    order.save()
    return order
```

3단계: 또 다른 실제 케이스

```
def test_create_order_with_invalid_user_id():
    order_data = {'user_id': -1, 'items': [{'id': 1}]}
    with pytest.raises(ValueError, match="user_id must be positive"):
        process_order(order_data)
```

```
def process_order(order_data: dict) -> Order:
    if not order_data.get('items'):
        raise ValueError("items cannot be empty")
```

```
    user_id = order_data.get('user_id')
    if user_id <= 0: # 실제 필요할 때 추가
        raise ValueError("user_id must be positive")
```

```
    order = Order(**order_data)
    order.save()
    return order
```



9. 리팩토링 시점의 차이

전통적인 구현

● 사후 리팩토링 (위험)

처음부터 복잡하게 작성

```
def calculate_price(items, user, coupon=None):
    total = 0
    for item in items:
        if item.on_sale:
            if user.is_premium:
                total += item.price * 0.7
            else:
                total += item.price * 0.9
        else:
            total += item.price

    if coupon:
        if coupon.type == 'percentage':
            total *= (1 - coupon.value / 100)
        elif coupon.type == 'fixed':
            total -= coupon.value

    if user.is_premium:
        total *= 0.95

    return max(total, 0)
```

- # 나중에 리팩토링하려면?
- # ✖ 테스트가 없어서 두려움
- # ✖ 어떤 동작이 의도된 것인지 불명확
- # ✖ 변경 시 버그 발생 위험

9. 리팩토링 시점의 차이

| GREEN 단계의 최소 구현

GREEN: 테스트 통과하는 코드

```
def test_calculate_price_basic():  
    items = [Item(price=100)]  
    user = User(is_premium=False)  
    assert calculate_price(items, user) == 100  
  
def calculate_price(items, user, coupon=None):  
    total = sum(item.price for item in items)  
    return total
```

9. 리팩토링 시점의 차이

| GREEN 단계의 최소 구현

RED: 새 요구사항

```
def test_calculate_price_with_sale():
    items = [Item(price=100, on_sale=True)]
    user = User(is_premium=False)
    assert calculate_price(items, user) == 90 # 10% 할인
```

GREEN: 기능 추가

```
def calculate_price(items, user, coupon=None):
    total = 0
    for item in items:
        if item.on_sale:
            total += item.price * 0.9
        else:
            total += item.price
    return total
```

9. 리팩토링 시점의 차이

| GREEN 단계의 최소 구현

● REFACTOR: 안전하게 개선

테스트가 있으므로 안전하게 리팩토링

```
def calculate_price(items, user, coupon=None):  
    total = sum(_calculate_item_price(item) for item in items)  
    return total
```

```
def _calculate_item_price(item):  
    base_price = item.price  
    if item.on_sale:  
        return base_price * 0.9  
    return base_price
```




10. 설계 진화 방식의 차이

전통적인 구현

Big Design Up Front (BDUF)

완벽한 설계 추구 (2주)

- 모든 클래스 다이어그램
- 모든 시퀀스 다이어그램
- 모든 데이터베이스 스키마
- 모든 API 명세



구현 (4주)

- 설계대로 구현
- 변경 어려움



테스트 & 버그 수정 (2주)

- 설계 변경 필요 발견
- 이미 늦음



10. 설계 진화 방식의 차이

GREEN 단계의 최소 구현

Emergent Design (창발적 설계)

Week 1:

테스트 → 구현 | Simple Class



Week 2:

테스트 → 구현 | Interface 추출



Week 3:

테스트 → 구현 | Strategy Pattern 적용



Week 4:

테스트 → 구현 | Repository Pattern 추가

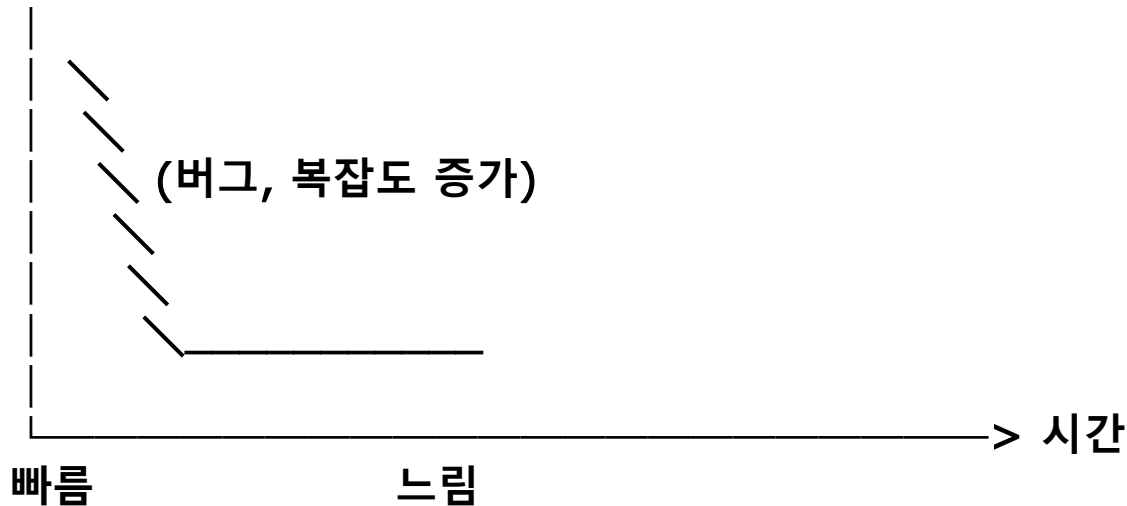
자연스럽게 좋은 설계로 진화

11. 개발 속도 비교

전통적인 구현

초기 빠름 → 후기 느림

개발 속도



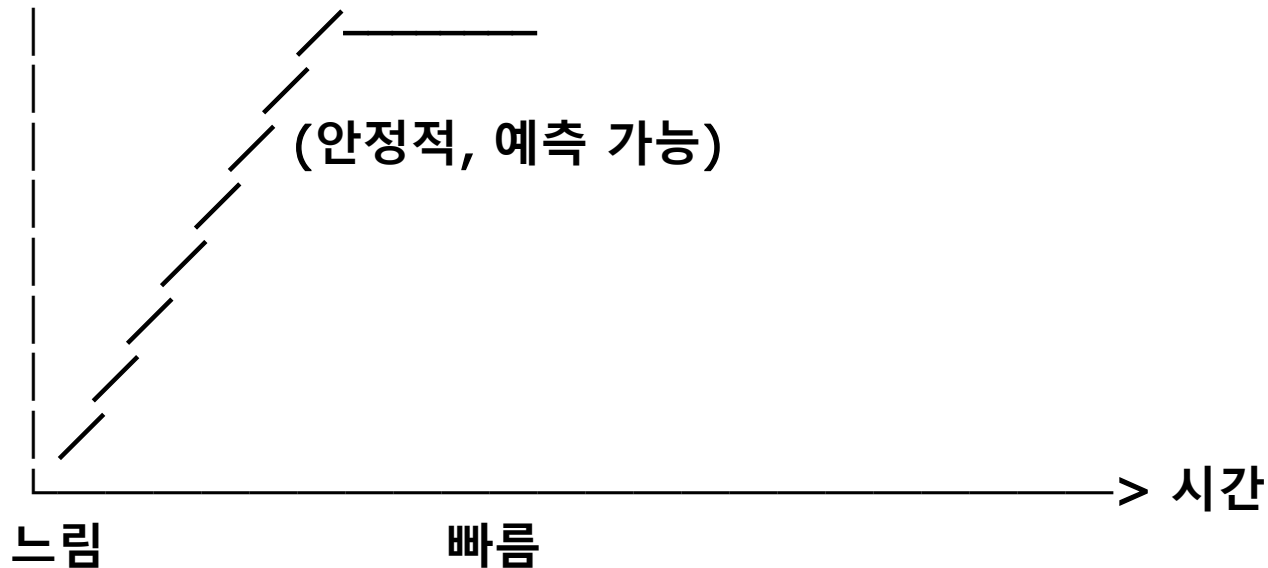
이유:

- 테스트 없어서 버그 발생
- 복잡한 코드 이해 어려움
- 변경 시 사이드 이펙트
- 리팩토링 두려움

11. 개발 속도 비교

GREEN 단계의 최소 구현

초기 느림 → 후기 빠름
개발 속도



이유:

- 테스트로 버그 조기 발견
- 리팩토링 자신감
- 코드 이해 쉬움
- 안전한 변경



12. “완성”의 정의

| 전통적인 구현에서의 “완성”

- 나 기준:
 - 내가 생각한 기능/시나리오를 코딩 다 했을 때
 - UI도 나왔고, 예외도 어느 정도 넣었고, 대충 돌아가면 완성
- 문제:
 - 테스트가 없거나 부족하면
 - → “완성됐다고 믿지만, 사실 잘못 동작하는 코드”일 수 있음



12. “완성”의 정의

GREEN 단계에서의 “완성”

- 명확한 기준:

- RED에서 정의한 테스트들이 전부 GREEN으로 바뀌었을 때
- 구현의 완료는 테스트 통과 여부에 의해 외부에서 정의됨
- 개발자의 “감”이 아니라 “테스트 스위트”가 기준

- 차이:

- 전통 구현의 완료는 개발자의 주관에 의존,
- GREEN 단계의 완료는 테스트에 의해 객관적으로 정의됩니다.

13. 실무 적용 가이드

| Step 1: 작게 시작하기

❌ 처음부터 완벽하게

```
def test_user_registration_with_all_validations():  
    # 20개의 검증을 한 번에 테스트
```

✅ 하나씩 테스트

```
def test_user_can_register_with_email():  
    user = register_user(email="test@example.com")  
    assert user.email == "test@example.com"
```



13. 실무 적용 가이드



Step 2: 가장 단순한 구현

❌ 처음부터 복잡하게

```
def register_user(email, password, name, phone, address, ...):  
    # 100줄의 검증 및 처리
```

✅ 최소 구현

```
def register_user(email):  
    return User(email=email)
```




13. 실무 적용 가이드

Step 3: 점진적 개선

다음 테스트

```
def test_user_password_is_hashed():  
    user = register_user(email="test@example.com", password="secret")  
    assert user.password != "secret"  
    assert bcrypt.check(user.password, "secret")
```

기능 추가

```
def register_user(email, password=None):  
    user = User(email=email)  
    if password:  
        user.password = bcrypt.hash(password)  
    return user
```

14. 실습

| 실습 2: 결제 시스템 구현

다음 전통적인(Traditional)방식으로 구현된 코드를 TDD GREEN 단계의 최소 구현 코드(GREEN)로 바꿔보세요.

구분	Traditional	GREEN
초기 구조	모든 기능을 한 번에 구현	테스트가 요구한 기능만 구현
복잡도	처음부터 복잡함	매우 단순
결합도	높음	낮음
확장성	변경 어려움	점진적으로 성장
테스트	나중에 추가 / 어려움	처음부터 테스트 기반

Week_2\Code_pro\python_test

14. 실습

실습 2: 결제 시스템 구현

```
# traditional_payment.py
from decimal import Decimal
```

```
class PaymentResult:
    def __init__(self, success: bool, amount: Decimal):
        self.success = success
        self.amount = amount

    def __repr__(self):
        return f"PaymentResult(success={self.success}, amount={self.amount})"
```

```
class TraditionalPaymentProcessor:
    """초기부터 모든 결제 방식을 한 클래스에 넣는 전통적 방식"""
```

```
    def process_payment(
        self,
        amount: Decimal,
        method: str,
        card_number: str = None,
        bank_account: str = None,
        crypto_wallet: str = None,
        paypal_email: str = None,
        apple_token: str = None,
        google_token: str = None
    ) -> PaymentResult:

        if method == "card":
            if not card_number:
                raise ValueError("Card number required")
            # (여기 100줄의 카드 결제 검증 로직이 있다고 가정)
            return PaymentResult(True, amount)
```



14. 실습

실습 2: 결제 시스템 구현

```
elif method == "bank_transfer":
    if not bank_account:
        raise ValueError("Bank account required")
    # (여기 80줄의 은행 이체 로직이 있다고 가정)
    return PaymentResult(True, amount)

elif method == "crypto":
    if not crypto_wallet:
        raise ValueError("Crypto wallet required")
    return PaymentResult(True, amount)

elif method == "paypal":
    if not paypal_email:
        raise ValueError("PayPal email required")
    return PaymentResult(True, amount)

elif method == "apple_pay":
    if not apple_token:
        raise ValueError("ApplePay token required")
    return PaymentResult(True, amount)

elif method == "google_pay":
    if not google_token:
        raise ValueError("GooglePay token required")
    return PaymentResult(True, amount)

else:
    raise ValueError(f"Unknown payment method: {method}")
```

14. 실습

실습 2: 결제 시스템 구현

```
# -----  
# 실행 테스트  
# -----  
if __name__ == "__main__":  
    processor = TraditionalPaymentProcessor()  
    result = processor.process_payment(  
        amount=Decimal("10000"),  
        method="card",  
        card_number="1111-2222-3333-4444"  
    )  
    print(result)
```



The screenshot shows a terminal window titled "Run" with a tab for "traditional_payment". The command executed is "C:\DEV\envs\py3_10_basic\Scripts\python.exe C:\DEV\SamsungDisplay\Week_2\Code_pro\python_test\traditional_payment.py". The output is "PaymentResult(success=True, amount=10000)" followed by a vertical bar. The terminal concludes with "Process finished with exit code 0".

```
Run traditional_payment x  
C:\DEV\envs\py3_10_basic\Scripts\python.exe C:\DEV\SamsungDisplay\Week_2\Code_pro\python_test\traditional_payment.py  
PaymentResult(success=True, amount=10000)  
|  
Process finished with exit code 0
```

『3과목』 최소구현 + 리뷰

- Part 1. 오리엔테이션
- Part 2. GREEN, Review Cursor AI 프롬프트
- Part 3. SW Implementation
- Part 4. GREEN 단계의 최소 구현
- Part 5. 종합 실습 & 마무리





1. Full-Stack GREEN + Review Cursor AI

최소 구현으로 테스트 통과 개발

Cursor AI 프롬프트 - GREEN 단계

컨텍스트

- 테스트 파일: [테스트 파일 경로]
- 구현 파일: [구현할 파일 경로]

테스트 코드

```
```typescript  
[전체 테스트 코드 붙여넣기]
```
```

요청

위 테스트를 ****모두 통과****하는 ****최소한의 구현 코드****를 작성해주세요.

요구사항

1. 테스트 통과에 필요한 코드만 작성
2. 과도한 최적화나 추가 기능 금지
3. TypeScript 타입 완전 정의
4. Props 인터페이스 명확히 정의

출력 형식

1. 타입 정의 (필요시)
2. 컴포넌트/함수 구현
3. CSS 스타일 (필요시)



1. Full-Stack GREEN + Review Cursor AI

실전 예시

컨텍스트

- 테스트 파일: frontend/src/components/customer/MenuCard.test.tsx
- 구현 파일: frontend/src/components/customer/MenuCard.tsx

테스트 코드

[MenuCard.test.tsx 전체 코드]

요청

모든 테스트를 통과하는 MenuCard.tsx를 구현해주세요.

추가 정보

- 스타일링: CSS 파일 분리 (MenuCard.css)
- 상태 관리: useState 사용
- 가격 계산: useMemo로 최적화



1. Full-Stack GREEN + Review Cursor AI

실전 예시 좀 더 구체화 ...

위 테스트를 모두 통과하는 MenuCard.tsx를 구현해주세요.



1. Full-Stack GREEN + Review Cursor AI

코드 리뷰 및 개선 포인트 도출

Cursor AI 프롬프트 - Review 단계

컨텍스트

- 구현 코드: [구현 파일 경로]
- 테스트 코드: [테스트 파일 경로]

구현 코드

```
```typescript
[구현 코드 붙여넣기]
```
```

요청

코드 리뷰를 진행하고 다음을 분석해주세요:

리뷰 관점

1. ****코드 품질****: 가독성, 유지보수성
2. ****성능****: 불필요한 렌더링, 메모리 누수
3. ****타입 안정성****: TypeScript 활용도
4. ****테스트 커버리지****: 누락된 테스트
5. ****접근성****: a11y 준수 여부
6. ****보안****: XSS, 입력 검증

출력 형식

| 영역 | 현재 상태 | 개선 제안 | 우선순위 |
|-------|-------|-------|----------|
| ----- | ----- | ----- | ----- |
| 성능 | ... | ... | P0/P1/P2 |

개선 포인트 코드 예시

각 제안에 대한 구체적인 코드 예시를 포함해주세요.



1. Full-Stack GREEN + Review Cursor AI

한 줄 프롬프트

GREEN 단계 한줄 프롬프트

- 위 테스트를 모두 통과하는 최소한의 구현 코드를 작성해주세요.
- 테스트 통과에 필요한 코드만 작성하고, 과도한 최적화나 추가 기능은 금지입니다.

Review 단계 한줄 프롬프트

- 이 코드를 리뷰해주세요. 코드 품질, 성능, 타입 안정성, 테스트 커버리지, 접근성, 보안 관점에서 분석하고, 개선 제안과 우선순위를 표로 정리해주세요.

2. 실전 예제: Frontend

사전 조건

RED 단계에서 작성된 테스트 코드가 있어야 합니다.

```
frontend/src/components/customer/MenuCard.test.tsx
import { describe, it, expect, vi } from 'vitest';
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { MenuCard } from './MenuCard';
```

```
const mockMenu = {
  id: '1',
  name: '아메리카노(ICE)',
  price: 4000,
  description: '시원한 아메리카노',
  imageUrl: '/images/americano.jpg',
  options: [
    { id: 'shot', label: '샷 추가', price: 500 },
    { id: 'syrup', label: '시럽 추가', price: 0 },
  ],
  stock: 10,
};

...

expect(mockOnAddToCart).toHaveBeenCalled();
});
});
});
```

2. 실전 예제: Frontend

GREEN 단계 - Cursor AI 프롬프트

Cursor AI에서 Cmd/Ctrl + I를 눌러 아래 프롬프트를 입력합니다:

Cursor AI 프롬프트 - GREEN 단계

컨텍스트

- 테스트 파일: frontend/src/components/customer/MenuCard.test.tsx
- 구현 파일: frontend/src/components/customer/MenuCard.tsx

테스트 코드

```
```typescript
// [위의 테스트 코드 전체 붙여넣기]
```
```

요청

모든 테스트를 통과하는 MenuCard.tsx를 구현해주세요.

추가 정보

- 스타일링: CSS 파일 분리 (MenuCard.css)
- 상태 관리: useState 사용
- 가격 계산: useMemo로 최적화

요구사항

1. 테스트 통과에 필요한 코드만 작성
2. 과도한 최적화나 추가 기능 금지
3. TypeScript 타입 완전 정의
4. Props 인터페이스 명확히 정의

2. 실전 예제: Frontend

GREEN 단계 - 예상 구현 결과

Cursor AI가 생성할 예상 코드:

```
frontend/src/components/customer/MenuCard.tsx
import React, { useState, useMemo } from 'react';
import './MenuCard.css';

interface MenuOption {
  id: string;
  label: string;
  price: number;
}

interface Menu {
  id: string;
  name: string;
  price: number;
  description: string;
  imageUrl?: string;
  options: MenuOption[];
  stock: number;
}...

    <button className="add-to-cart-button" onClick={handleAddToCart}>
      담기
    </button>
  </div>
);
};
```

2. 실전 예제: Frontend

테스트 실행 및 확인

```
cd frontend  
npm test -- --run
```

예상 결과:

```
k8s@DESKTOP-R0EQ2U6 MINGW64 /c/DEV/SamsungDisplay/Week_2/Cursor_pro/OrderBean/fro  
ntend (backend)  
$ npm test -- --run  
  
> frontend@0.0.0 test  
> vitest --run  
  
RUN v4.0.6 C:/DEV/SamsungDisplay/Week_2/Cursor_pro/OrderBean/frontend  
  
> src/components/admin/OrdersSection.test.tsx (9 tests | 8 failed) 56ms  
  x 섹션 제목 "주문 현황"을 표시한다 14ms  
  x 주문 리스트를 표시한다 2ms  
  x 주문 일자와 시간을 표시한다 2ms  
  x 주문 금액을 표시한다 1ms  
  x pending 상태일 때 "주문 접수" 버튼을 표시한다 6ms  
  x preparing 상태일 때 "제조 완료" 버튼을 표시한다 3ms  
  x "주문 접수" 버튼 클릭 시 상태가 preparing으로 변경된다 2ms  
  x "제조 완료" 버튼 클릭 시 상태가 ready로 변경된다 7ms  
  ✓ 주문이 없을 때 빈 상태 메시지를 표시한다 19ms  
  
> src/components/admin/InventorySection.test.tsx (9 tests | 6 failed) 449ms
```

2. 실전 예제: Frontend

Review 단계 - Cursor AI 프롬프트

테스트가 모두 통과하면, Review 단계로 진행합니다:

Cursor AI 프롬프트 - Review 단계

컨텍스트

- 구현 코드: frontend/src/components/customer/MenuCard.tsx
- 테스트 코드: frontend/src/components/customer/MenuCard.test.tsx

구현 코드

```
```typescript
// [위에서 구현한 MenuCard.tsx 전체 코드 붙여넣기]
```
```

요청

코드 리뷰를 진행하고 다음을 분석해주세요:

리뷰 관점

1. **코드 품질**: 가독성, 유지보수성
2. **성능**: 불필요한 렌더링, 메모리 누수
3. **타입 안정성**: TypeScript 활용도
4. **테스트 커버리지**: 누락된 테스트
5. **접근성**: a11y 준수 여부
6. **보안**: XSS, 입력 검증

출력 형식

| 영역 | 현재 상태 | 개선 제안 | 우선순위 |
|----|-------|-------|----------|
| 성능 | ... | ... | P0/P1/P2 |

개선 포인트 코드 예시

각 제안에 대한 구체적인 코드 예시를 포함해주세요.

2. 실전 예제: Frontend

Review 단계 - 예상 결과

Cursor AI가 생성할 예상 리뷰:

코드 리뷰 결과

| 영역 | 현재 상태 | 개선 제안 | 우선순위 |
|----------|-------|--------------------------------------|------|
| 코드 품질 | 양호 | 타입을 별도 파일로 분리 | P2 |
| 성능 | 양호 | handleOptionChange를 useCallback으로 래핑 | P1 |
| 타입 안정성 | 양호 | Menu 타입을 types/menu.ts로 이동 | P2 |
| 테스트 커버리지 | 부족 | 옵션 없는 메뉴, 재고 0 케이스 추가 필요 | P1 |
| 접근성 | 부족 | aria-label 추가 필요 | P1 |
| 보안 | 양호 | XSS 위험 없음 | - |

개선 포인트 상세

1. 성능 개선 (P1)

```typescript

// 현재

```
const handleOptionChange = (optionId: string) => { ... };
```

// 개선

```
const handleOptionChange = useCallback((optionId: string) => {
 setSelectedOptions(prev =>
 prev.includes(optionId)
 ? prev.filter(id => id !== optionId)
 : [...prev, optionId]
 });
}, []);
```



## 2. 실전 예제: Frontend

### Review 단계 - 예상 결과

#### 2. 접근성 개선 (P1)

```
```typescript
// 현재
```

```
// 개선
```

```
```
```

#### 3. 누락된 테스트 케이스 (P1)

```
```typescript
// 추가 필요한 테스트
it('옵션이 없는 메뉴를 정상 표시한다', () => { ... });
it('재고가 0이면 담기 버튼이 비활성화된다', () => { ... });
it('이미지가 없으면 플레이스홀더를 표시한다', () => { ... });
```
```



# 3. 실전 예제: Backend API

## GREEN 단계 - Backend 프롬프트

### # Cursor AI 프롬프트 - GREEN 단계 (Backend)

## 컨텍스트

- 테스트 파일: backend/tests/test\_menus.py
- 구현 파일: backend/app/api/v1/menus.py

## 테스트 코드

```
```python
import pytest
from httpx import AsyncClient
```

@pytest.mark.asyncio

class TestMenuAPI:

async def test_get_menus_returns_list(self, async_client: AsyncClient):

"""GET /api/v1/menus - 메뉴 목록을 반환한다"""

response = await async_client.get("/api/v1/menus")

assert response.status_code == 200

assert isinstance(response.json(), list)

async def test_get_menu_by_id(self, async_client: AsyncClient, sample_menu):

"""GET /api/v1/menus/{id} - 특정 메뉴를 반환한다"""

response = await async_client.get(f"/api/v1/menus/{sample_menu.id}")

assert response.status_code == 200

assert response.json()["name"] == sample_menu.name

async def test_get_menu_not_found(self, async_client: AsyncClient):

"""없는 메뉴는 404를 반환한다"""

response = await async_client.get("/api/v1/menus/99999")

assert response.status_code == 404



3. 실전 예제: Backend API

GREEN 단계 - Backend 프롬프트

```
async def test_create_menu(self, async_client: AsyncClient):
    """POST /api/v1/menus - 메뉴를 생성한다"""
    menu_data = {"name": "아메리카노", "price": 4000}
    response = await async_client.post("/api/v1/menus", json=menu_data)
    assert response.status_code == 201
...
```

요청

위 테스트를 모두 통과하는 FastAPI 엔드포인트를 구현해주세요.

구현할 파일

1. backend/app/api/v1/menus.py - API 라우터
2. backend/app/schemas/menu.py - Pydantic 스키마
3. backend/app/models/menu.py - SQLAlchemy 모델

요구사항

1. 테스트 통과에 필요한 코드만 작성
2. 비동기 SQLAlchemy 사용
3. 적절한 HTTP 상태 코드 반환



3. 실전 예제: Backend API

Review 단계 - Backend 프롬프트

Cursor AI 프롬프트 - Review 단계 (Backend)

컨텍스트

- 구현 코드: backend/app/api/v1/menus.py

구현 코드

```python

[menus.py 코드 붙여넣기]

```

요청

코드 리뷰를 진행하고 다음을 분석해주세요:

리뷰 관점

1. **코드 품질**: 가독성, 유지보수성
2. **성능**: N+1 쿼리, 인덱싱
3. **에러 처리**: 예외 처리 일관성
4. **타입 안정성**: Pydantic 활용도
5. **보안**: SQL 인젝션, 입력 검증
6. **테스트 커버리지**: 누락된 케이스

출력 형식

영역	현재 상태	개선 제안	우선순위



4. 체크리스트

| GREEN 단계 체크리스트

- ☐ 테스트 코드 전체를 Cursor AI에 제공했는가?
- ☐ 타입/인터페이스가 명확히 정의되었는가?
- ☐ 모든 테스트가 통과하는가?
- ☐ 테스트에 없는 기능을 추가하지 않았는가?
- ☐ 과도한 최적화를 하지 않았는가?

4. 체크리스트

Review 단계 체크리스트

- ☐ 구현 코드 전체를 Cursor AI에 제공했는가?
- ☐ 6가지 리뷰 관점을 모두 검토했는가?
 - ☐ 코드 품질
 - ☐ 성능
 - ☐ 타입 안정성
 - ☐ 테스트 커버리지
 - ☐ 접근성
 - ☐ 보안
- ☐ 개선 포인트에 우선순위를 지정했는가?
- ☐ 개선 포인트에 코드 예시가 포함되었는가?
- ☐ P0 이슈가 있다면 즉시 수정 계획을 세웠는가?



4. 체크리스트

다음 단계로 진행 조건

GREEN 단계 완료 조건:

- 모든 테스트 통과
- 테스트에 필요한 최소한의 코드만 작성

Review 단계 완료 조건:

- 6가지 관점에서 리뷰 완료
- 개선 포인트 목록 작성
- 우선순위 지정 완료

→ 6단계 Refactor로 진행

5. 오늘 배운 내용 정리

핵심 요약

이론 (2시간)

- 소프트웨어 설계 3요소: ERD, API, 아키텍처
- NCS 소프트웨어 개발 표준 준수
- SOLID 원칙 실무 적용
- Layered Architecture 구조
- 설계 품질 기준 (결합도↓, 응집도↑)

실습 (5시간)

- ERD 자동 생성 (Mermaid + DDL + 마이그레이션)
- API 문서 자동화 (OpenAPI + Swagger + Postman Collection)
- 아키텍처 다이어그램 (C4 Model 4단계 + 시퀀스)
- 설계-코드 일치 검증 (87% 달성)
- 통합 문서 사이트 (MkDocs + GitHub Pages)
- CI/CD 자동화 (GitHub Actions)



5. 오늘 배운 내용 정리

실무 산출물

40페이지 문서 패키지
자동 업데이트 파이프라인
검증 보고서 + 액션 아이템
팀 협업 워크플로우

6. 실무 적용 가이드

| 실전 투입 체크리스트

문서화 기반 구축

- 프로젝트에 MkDocs 설정
- README.md 템플릿 작성
- ERD 자동 생성 스크립트 설정
- GitHub Actions 기본 파이프라인 구축

API 문서 자동화

- Swagger UI 커스터마이징
- OpenAPI 명세 상세화
- Postman Collection 생성
- API 테스트 시나리오 작성



6. 실무 적용 가이드

| 실전 투입 체크리스트

아키텍처 문서화

- C4 Model 다이어그램 작성
- 시퀀스 다이어그램 (핵심 플로우)
- 배포 아키텍처 문서
- 기술 의사결정 기록 (ADR)

품질 고도화

- 설계-코드 검증 자동화
- 문서 Lint 설정
- 테스트 커버리지 향상 (85% 이상)
- 팀 리뷰 프로세스 확립

6. 실무 적용 가이드

| 투자 대비 효과

초기 투자: 4주 (인력 1명)

연간 절감:

- 신입 온보딩 시간: 50% 단축 (8주 → 4주)
- 문서 작성 시간: 80% 단축 (주 5시간 → 1시간)
- 버그 수정 시간: 30% 단축 (설계 이해 향상)
- 의사소통 오류: 40% 감소

ROI: 1년 내 300% 이상



7. 다음 단계 (Day 4 Preview)



DAY 4: 리팩토링 + 코드 리뷰 + CI/CD 배포
Full-Stack Optional + Cursor AI 개발

THANK YOU.

앞으로의 엔지니어는 단순한 '코더'나 '기계 조작자'가 아니라 뇌-기계 인터페이스를 통해 지식과 능력을 즉각 확장하는 존재(뉴로-인터페이스: Neuro Interface)가 될 수 있습니다.

- 🎯 목표 달성을 위한 여정이 시작됩니다.
- 🌟 궁금한 점이 있으시면 언제든지 문의해주세요!
- 🚀 함께 코더와 프롬프트 전문가로 성장해 나갑시다!